Winter Cup

Competitive Programming Handbook

ICPC-Style Contest Reference

Theorems · Algorithms · Implementations · Proofs

Contents:

§1 Number Theory
§2 Combinatorics
§3 Graph Theory
§4 Dynamic Programming
§5 Greedy Algorithms
§6 Data Structures
§7 String Algorithms
§8 Geometry
§9 Probability & Math
§10 CP Patterns

All implementations in Python 3 · Contest-ready · PDF printable

Compiled for ICPC / ACM-style contests

Contents

1 Number Theory **3**
1.1 Fermat's Little Theorem . . . 3
1.2 Euler's Totient Theorem . . . 3
1.3 Modular Inverse . . . 4
1.4 Extended Euclidean Algorithm . . . 5
1.5 Chinese Remainder Theorem (CRT) . . . 5
1.6 Wilson's Theorem . . . 6
1.7 Sieve of Eratosthenes . . . 7
1.8 Prime Factorization . . . 8
1.9 GCD / LCM . . . 8
1.10 Fast Exponentiation (Binary Exponentiation) . . . 9
1.11 Diophantine Equations . . . 10

2 Combinatorics **11**
2.1 Binomial Theorem . . . 11
2.2 Binomial Coefficients & Precomputed Factorials . . . 11
2.3 Stars and Bars . . . 12
2.4 Inclusion–Exclusion Principle . . . 12
2.5 Pigeonhole Principle . . . 13
2.6 Catalan Numbers . . . 13
2.7 Sperner's Theorem . . . 14
2.8 Derangements . . . 15
2.9 Combinations with Repetition . . . 15

3 Graph Theory **16**
3.1 BFS – Breadth-First Search . . . 16
3.2 DFS – Depth-First Search . . . 16
3.3 Dijkstra's Algorithm . . . 17
3.4 Bellman-Ford Algorithm . . . 18
3.5 Floyd-Warshall Algorithm . . . 18
3.6 Topological Sort . . . 19
3.7 Kruskal's Algorithm (MST) . . . 20
3.8 Prim's Algorithm (MST) . . . 20
3.9 Union-Find (DSU) . . . 21
3.10 Tarjan's SCC . . . 21
3.11 Kosaraju's SCC . . . 22
3.12 Bridges and Articulation Points . . . 23
3.13 LCA with Binary Lifting . . . 24
3.14 Euler Tour . . . 24
3.15 Maximum Flow – Dinic's Algorithm . . . 25
3.16 Bipartite Matching (Hopcroft-Karp) . . . 26
3.17 Minimum Cut . . . 26
3.18 Bipartite Graph Theorem . . . 27

4 Dynamic Programming **29**
4.1 0/1 Knapsack . . . 29
4.2 Unbounded Knapsack . . . 29
4.3 Coin Change . . . 29
4.4 LIS – Longest Increasing Subsequence $\mathcal{O}(n \log n)$ . . . 30
4.5 LCS – Longest Common Subsequence . . . 30
4.6 Interval DP . . . 31
4.7 Bitmask DP . . . 32
4.8 Digit DP . . . 32
4.9 Tree DP . . . 33
4.10 DP with Prefix Sums . . . 33
4.11 Meet-in-the-Middle DP . . . 34
4.12 State Compression DP . . . 34
4.13 DP Optimizations . . . 34

5 Greedy Algorithms **36**
5.1 Interval Scheduling / Activity Selection . . . 36
5.2 Huffman Coding . . . 36
5.3 Greedy Exchange Argument . . . 37
5.4 Fractional Knapsack . . . 37

6 Data Structures **38**
6.1 Segment Tree . . . 38
6.2 Lazy Propagation . . . 38
6.3 Fenwick Tree (BIT) . . . 39
6.4 Sparse Table . . . 40
6.5 Trie . . . 41
6.6 Monotonic Stack . . . 43
6.7 Monotonic Queue (Deque) . . . 43
6.8 Priority Queue (Heap) . . . 43
6.9 Hash Map / Hashing . . . 44

7 String Algorithms **46**
7.1 KMP Algorithm . . . 46
7.2 Z-Algorithm . . . 46
7.3 Suffix Array . . . 47
7.4 Rolling Hash / Rabin-Karp . . . 47
7.5 Manacher's Algorithm . . . 48
7.6 Trie-based Matching . . . 48

8 Geometry **50**
8.1 Cross Product & Dot Product . . . 50
8.2 Convex Hull (Andrew's Monotone Chain) . . . 50
8.3 Shoelace Formula & Polygon Area . . . 51
8.4 Pick's Theorem . . . 51
8.5 Line Intersection . . . 51
8.6 Point in Polygon . . . 52
8.7 Distance Formulas . . . 53

9 Probability & Mathematics **54**
9.1 Expected Value & Linearity of Expectation . . . 54
9.2 Bayes' Theorem . . . 54
9.3 Randomized Algorithms . . . 55
9.4 Randomized Techniques in CP . . . 56

10 CP Patterns **57**
10.1 Sliding Window . . . 57
10.2 Two Pointers . . . 57
10.3 Binary Search on Answer . . . 58
10.4 Prefix Sums . . . 58
10.5 Difference Arrays . . . 59
10.6 Coordinate Compression . . . 59
10.7 Mo's Algorithm . . . 60
10.8 Offline Queries . . . 60
10.9 Bitmasking . . . 61
10.10 Subset Iteration Trick (SOS DP) . . . 61
10.11 Meet-in-the-Middle . . . 62
10.12 Fast I/O Template . . . 62

1 Number Theory

1.1 Fermat's Little Theorem

Theorem 1.1 (Fermat's Little Theorem)**.** Let p be a prime and a an integer with $\gcd(a, p) = 1$. Then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Equivalently, $a^p \equiv a \pmod{p}$ for every integer a.

Intuition. When you raise a to the power $p - 1$ modulo a prime p, the group structure of $(\mathbb{Z}/p\mathbb{Z})^*$ forces the result to collapse to 1.

CP Utility: Fast computation of modular inverses: $a^{-1} \equiv a^{p-2} \pmod{p}$ when p is prime.

Where to use.

- Computing $\binom{n}{k}$ mod p (modular inverse of factorials).
- Reducing huge exponents: $a^x \equiv a^{x \bmod (p-1)} \pmod{p}$.
- Hashing, cryptography problems.

Typical CP scenarios. Counting paths, number of ways, divide-and-conquer with modular arithmetic.

Example Problem. Compute $\binom{n}{k}$ mod p for large n, k where p is prime.

1.1.1 Python Implementation

```python
MOD = 10**9 + 7

def mod_inv(a, p=MOD):
    # Fermat: a^(p-2) mod p = a^(-1) mod p
    return pow(a, p - 2, p)

def comb_mod(n, k, p=MOD):
    if k < 0 or k > n:
        return 0
    num = 1
    den = 1
    for i in range(k):
        num = num * (n - i) % p
        den = den * (i + 1) % p
    return num * mod_inv(den, p) % p

# Example
print(comb_mod(10, 3))  # C(10,3) mod 10^9+7 = 120
```

Listing 1: Fermat's Little Theorem – Modular Inverse

Complexity. $\mathcal{O}(\log p)$ per inverse via fast exponentiation.

Note / Pitfall: Only valid when p is prime. For composite moduli use the Extended Euclidean Algorithm instead.

1.2 Euler's Totient Theorem

Theorem 1.2 (Euler's Totient Theorem)**.** For $\gcd(a, n) = 1$:

$$a^{\phi(n)} \equiv 1 \pmod{n},$$

where $\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$ counts integers in $[1, n]$ coprime to n.

Intuition. Generalizes Fermat to composite moduli. The order of a in $(\mathbb{Z}/n\mathbb{Z})^*$ divides $\phi(n)$.

Mathematical formulation.

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right), \quad \phi(p^k) = p^{k-1}(p-1).$$

CP Utility: Reduces exponents modulo $\phi(n)$; useful when the modulus is not prime.

1.2.1 Python Implementation

```python
def euler_phi(n):
    result = n
    p = 2
    temp = n
    while p * p <= temp:
        if temp % p == 0:
            while temp % p == 0:
                temp //= p
            result -= result // p
        p += 1
    if temp > 1:
        result -= result // temp
    return result

# Sieve version for range [1, N]
def phi_sieve(N):
    phi = list(range(N + 1))
    for i in range(2, N + 1):
        if phi[i] == i:          # i is prime
            for j in range(i, N + 1, i):
                phi[j] -= phi[j] // i
    return phi

print(euler_phi(12))  # 4: {1,5,7,11}
```

Listing 2: Euler's Totient Function

Complexity. $\mathcal{O}(\sqrt{n})$ per value; $\mathcal{O}(n \log \log n)$ for sieve.

Note / Pitfall: Euler's theorem requires $\gcd(a, n) = 1$. If not, the formula breaks.

1.3 Modular Inverse

Definition 1.3. The modular inverse of a modulo m is x such that $ax \equiv 1 \pmod{m}$. It exists iff $\gcd(a, m) = 1$.

Three methods: (1) Fermat (m prime), (2) Extended GCD (general), (3) Linear sieve for range.

1.3.1 Python Implementation

```python
MOD = 10**9 + 7

# Method 1: Fermat (m must be prime)
def inv_fermat(a, m=MOD):
    return pow(a, m - 2, m)

# Method 2: Extended GCD
def ext_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = ext_gcd(b, a % b)
    return g, y, x - (a // b) * y

def inv_ext(a, m):
    g, x, _ = ext_gcd(a % m, m)
```

```python
    if g != 1:
        return None  # No inverse
    return x % m

# Method 3: Linear sieve for inv[1..N]
def inv_sieve(N, m=MOD):
    inv = [0] * (N + 1)
    inv[1] = 1
    for i in range(2, N + 1):
        inv[i] = -(m // i) * inv[m % i] % m
    return inv
```

Listing 3: Modular Inverse – three methods

Complexity. Fermat: $\mathcal{O}(\log m)$; Ext GCD: $\mathcal{O}(\log m)$; Sieve: $\mathcal{O}(N)$.

1.4 Extended Euclidean Algorithm

Theorem 1.4 (Bézout's Identity)**.** For integers a, b (not both zero), there exist integers x, y such that

$$ax + by = \gcd(a, b).$$

The Extended Euclidean Algorithm computes $\gcd(a, b)$, x, y simultaneously.

Intuition. Backtrack the Euclidean steps to express the GCD as a linear combination.

CP Utility: Solve linear Diophantine equations; compute modular inverses for non-prime moduli; CRT.

1.4.1 Python Implementation

```python
def ext_gcd(a, b):
    """Returns (gcd, x, y) such that a*x + b*y = gcd."""
    if b == 0:
        return a, 1, 0
    g, x1, y1 = ext_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

# Iterative version (avoids recursion limit)
def ext_gcd_iter(a, b):
    old_r, r = a, b
    old_s, s = 1, 0
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    # old_s * a + ((old_r - old_s * a) // b) * b = old_r
    y = (old_r - old_s * a) // b if b else 0
    return old_r, old_s, y

g, x, y = ext_gcd(35, 15)
print(g, x, y)  # 5, 1, -2  => 35*1 + 15*(-2) = 5
```

Listing 4: Extended Euclidean Algorithm

Complexity. $\mathcal{O}(\log(\min(a, b)))$.

1.5 Chinese Remainder Theorem (CRT)

Theorem 1.5 (Chinese Remainder Theorem)**.** Let $m_1, m_2, \ldots, m_k$ be pairwise coprime moduli. Then the system

$$x \equiv a_i \pmod{m_i}, \quad i = 1, \ldots, k$$

has a unique solution modulo $M = m_1 m_2 \cdots m_k$.

Intuition. The mapping $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a ring isomorphism $\mathbb{Z}/M\mathbb{Z} \to \mathbb{Z}/m_1\mathbb{Z} \times \cdots \times \mathbb{Z}/m_k\mathbb{Z}$.

CP Utility: Combine partial results from separate moduli; reconstruct large answers; speed up DP via modular reduction.

1.5.1 Python Implementation

```python
def crt(remainders, moduli):
    """CRT for pairwise coprime moduli."""
    M = 1
    for m in moduli:
        M *= m
    x = 0
    for a, m in zip(remainders, moduli):
        Mi = M // m
        _, inv, _ = ext_gcd(Mi, m)
        x += a * Mi * inv
    return x % M

def ext_gcd(a, b):
    if b == 0: return a, 1, 0
    g, x, y = ext_gcd(b, a % b)
    return g, y, x - (a // b) * y

# General CRT: merge two equations x = a1 (mod m1), x = a2 (mod m2)
def crt2(a1, m1, a2, m2):
    g, p, _ = ext_gcd(m1, m2)
    if (a2 - a1) % g != 0:
        return None, None  # No solution
    lcm = m1 // g * m2
    ans = (a1 + m1 * ((a2 - a1) // g * p % (m2 // g))) % lcm
    return ans, lcm

# Example: x=2(mod 3), x=3(mod 5), x=2(mod 7) => x=23
print(crt([2, 3, 2], [3, 5, 7]))  # 23
```

Listing 5: CRT – pairwise coprime and general version

Complexity. $\mathcal{O}(k \log M)$.

1.6 Wilson's Theorem

Theorem 1.6 (Wilson's Theorem)**.** An integer $p > 1$ is prime if and only if

$$(p-1)! \equiv -1 \pmod{p}.$$

Intuition. In $(\mathbb{Z}/p\mathbb{Z})^*$, every element pairs with its inverse except ± 1. So the product of all non-zero residues is $(-1)(1) = -1$.

CP Utility: Primality test (theoretical); computing $(p-1)! \bmod p$ directly; generating problems with factorial modular structure.

1.6.1 Python Implementation

```python
def wilson_prime(p):
    """Returns True if p is prime (Wilson's Theorem).
    Not efficient for large p -- use as theoretical check."""
    if p < 2:
        return False
    factorial = 1
    for i in range(1, p):
        factorial = factorial * i % p
    return factorial == p - 1
```

```python

# In practice: (p-1)! mod p for prime p = p-1
print(wilson_prime(7))    # True
print(wilson_prime(10))   # False
```

Listing 6: Wilson's Theorem – primality test

Complexity. $\mathcal{O}(p)$ – only theoretical use, not practical for primality.

Note / Pitfall: Wilson's theorem is rarely used directly in contests but appears in problems asking for $(n-1)! \bmod n$ or detecting primes algebraically.

1.7 Sieve of Eratosthenes

Algorithm 1.7 (Sieve of Eratosthenes)**.** Mark all integers ≥ 2 as prime. For each prime p found, mark all multiples $p^2, p^2+p, \ldots$ as composite.

Intuition. Any composite $n \leq N$ has a prime factor $\leq \sqrt{N}$, so crossing off multiples of primes up to $\sqrt{N}$ is sufficient.

CP Utility: Precompute all primes up to N; smallest prime factor (SPF) for factorization; Euler's totient sieve.

1.7.1 Python Implementation

```python
def sieve(N):
    """Returns boolean array: is_prime[i] = True if i is prime."""
    is_prime = bytearray([1]) * (N + 1)
    is_prime[0] = is_prime[1] = 0
    for i in range(2, int(N**0.5) + 1):
        if is_prime[i]:
            is_prime[i*i::i] = bytearray(len(is_prime[i*i::i]))
    return is_prime

def spf_sieve(N):
    """Smallest prime factor for each i in [2, N]."""
    spf = list(range(N + 1))
    for i in range(2, int(N**0.5) + 1):
        if spf[i] == i:  # i is prime
            for j in range(i*i, N + 1, i):
                if spf[j] == j:
                    spf[j] = i
    return spf

def factorize_fast(n, spf):
    """Factorize n using precomputed SPF in O(log n)."""
    factors = []
    while n > 1:
        p = spf[n]
        while n % p == 0:
            factors.append(p)
            n //= p
    return factors

primes = [i for i, v in enumerate(sieve(50)) if v]
print(primes)  # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Listing 7: Sieve of Eratosthenes + Smallest Prime Factor sieve

Complexity. $\mathcal{O}(N \log\log N)$ time; $\mathcal{O}(N)$ space.

1.8 Prime Factorization

Algorithm 1.8 (Trial Division). To factorize n: try all primes $p \leq \sqrt{n}$. If $p \mid n$, divide repeatedly. Any remaining factor > 1 is prime.

1.8.1 Python Implementation

```python
def factorize(n):
    """Returns dict {prime: exponent}."""
    factors = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            factors[d] = factors.get(d, 0) + 1
            n //= d
        d += 1
    if n > 1:
        factors[n] = factors.get(n, 0) + 1
    return factors

def num_divisors(n):
    f = factorize(n)
    result = 1
    for exp in f.values():
        result *= (exp + 1)
    return result

def sum_divisors(n):
    f = factorize(n)
    result = 1
    for p, e in f.items():
        result *= (p**(e+1) - 1) // (p - 1)
    return result

print(factorize(360))  # {2:3, 3:2, 5:1}
print(num_divisors(360))  # 24
```

Listing 8: Prime Factorization by Trial Division

Complexity. $\mathcal{O}(\sqrt{n})$ per number; $\mathcal{O}(\log n)$ with precomputed SPF.

1.9 GCD / LCM

Theorem 1.9 (Euclidean Algorithm). $\gcd(a, b) = \gcd(b, a \bmod b)$, with $\gcd(a, 0) = a$.

$$\operatorname{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}.$$

1.9.1 Python Implementation

```python
from math import gcd
from functools import reduce

def lcm(a, b):
    return a // gcd(a, b) * b

def gcd_list(lst):
    return reduce(gcd, lst)

def lcm_list(lst):
    return reduce(lcm, lst)

# GCD of array
print(gcd_list([12, 18, 24]))  # 6
```

```python
print(lcm_list([4, 6, 10]))     # 60
```

Listing 9: GCD and LCM

Complexity. $\mathcal{O}(\log(\min(a, b)))$.

1.10 Fast Exponentiation (Binary Exponentiation)

Algorithm 1.10 (Binary Exponentiation)**.** Compute $a^b \bmod m$ in $\mathcal{O}(\log b)$ by repeated squaring:

$$a^b = \begin{cases} 1 & b = 0 \\ (a^{b/2})^2 & b \text{ even} \\ a \cdot a^{b-1} & b \text{ odd} \end{cases}$$

1.10.1 Python Implementation

```python
def power(base, exp, mod):
    """Compute base^exp mod (mod)."""
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = result * base % mod
        base = base * base % mod
        exp >>= 1
    return result

# Matrix exponentiation
def mat_mul(A, B, mod):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            if A[i][k] == 0: continue
            for j in range(n):
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod
    return C

def mat_pow(M, p, mod):
    n = len(M)
    result = [[int(i==j) for j in range(n)] for i in range(n)]
    while p:
        if p & 1:
            result = mat_mul(result, M, mod)
        M = mat_mul(M, M, mod)
        p >>= 1
    return result

# Fibonacci via matrix exponentiation
def fib(n, mod=10**9+7):
    if n <= 1: return n
    M = [[1,1],[1,0]]
    return mat_pow(M, n-1, mod)[0][0]

print(power(2, 100, 10**9+7))
print(fib(50))
```

Listing 10: Binary Exponentiation – matrix and modular versions

Complexity. $\mathcal{O}(\log b)$ for scalar; $\mathcal{O}\big(k^3 \log b\big)$ for $k \times k$ matrix.

1.11 Diophantine Equations

Theorem 1.11. The linear Diophantine equation $ax + by = c$ has integer solutions iff $\gcd(a, b) \mid c$. If (x_0, y_0) is one solution, the general solution is:

$$x = x_0 + \frac{b}{d}t, \quad y = y_0 - \frac{a}{d}t, \quad t \in \mathbb{Z},$$

where $d = \gcd(a, b)$.

1.11.1 Python Implementation

```python
def ext_gcd(a, b):
    if b == 0: return a, 1, 0
    g, x, y = ext_gcd(b, a % b)
    return g, y, x - (a // b) * y

def diophantine(a, b, c):
    """Solve ax + by = c. Returns (x0, y0, step_x, step_y) or None."""
    g, x0, y0 = ext_gcd(abs(a), abs(b))
    if c % g != 0:
        return None  # No integer solution
    x0 *= c // g
    y0 *= c // g
    if a < 0: x0 = -x0
    if b < 0: y0 = -y0
    return x0, y0, b // g, a // g
    # General: x = x0 + (b/g)*t, y = y0 - (a/g)*t

sol = diophantine(3, 5, 11)
if sol:
    x0, y0, dx, dy = sol
    print(f"x={x0} + {dx}*t, y={y0} - {dy}*t")
```

Listing 11: Linear Diophantine Equation solver

Complexity. $\mathcal{O}(\log(\min(a, b)))$.

Note / Pitfall: For minimizing $|x| + |y|$ or finding positive solutions, iterate t to constrain the solution.

2 Combinatorics

2.1 Binomial Theorem

Theorem 2.1 (Binomial Theorem). For integers $n \geq 0$ and any a, b:

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}.$$

Intuition. Each term in the expansion picks either a or b from each factor; $\binom{n}{k}$ counts the ways to choose k copies of a.

CP Utility: Expanding polynomial expressions; computing sums like $\sum_k \binom{n}{k} x^k$; generating functions.

2.1.1 Python Implementation

```python
MOD = 10**9 + 7

def binomial_expansion_coeff(n, target_power_a):
    """Returns coefficient of a^k b^(n-k) in (a+b)^n."""
    k = target_power_a
    return comb_mod(n, k)

def comb_mod(n, k, mod=MOD):
    if k < 0 or k > n: return 0
    if k == 0 or k == n: return 1
    # Precompute factorials
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i-1] * i % mod
    inv_fact = [1] * (n + 1)
    inv_fact[n] = pow(fact[n], mod - 2, mod)
    for i in range(n - 1, -1, -1):
        inv_fact[i] = inv_fact[i+1] * (i+1) % mod
    return fact[n] * inv_fact[k] % mod * inv_fact[n-k] % mod
```

Listing 12: Binomial expansion and coefficient extraction

Complexity. $\mathcal{O}(n)$ precomputation, $\mathcal{O}(1)$ per query.

2.2 Binomial Coefficients & Precomputed Factorials

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

2.2.1 Python Implementation

```python
class Combinatorics:
    def __init__(self, N, mod=10**9+7):
        self.mod = mod
        self.fact = [1] * (N + 1)
        self.inv_fact = [1] * (N + 1)
        for i in range(1, N + 1):
            self.fact[i] = self.fact[i-1] * i % mod
        self.inv_fact[N] = pow(self.fact[N], mod - 2, mod)
        for i in range(N - 1, -1, -1):
            self.inv_fact[i] = self.inv_fact[i+1] * (i+1) % mod

    def C(self, n, k):
        if k < 0 or k > n: return 0
        return self.fact[n] * self.inv_fact[k] % self.mod * self.inv_fact[n-k]
            % self.mod

```

```python
    def P(self, n, k):
        """Permutations P(n,k) = n!/(n-k)!"""
        if k < 0 or k > n: return 0
        return self.fact[n] * self.inv_fact[n-k] % self.mod

cb = Combinatorics(200005)
print(cb.C(100, 3))    # 161700
print(cb.P(10, 3))     # 720
```

Listing 13: Factorial precomputation for fast C(n,k) mod p

2.3 Stars and Bars

Theorem 2.2 (Stars and Bars)**.** The number of ways to place n indistinguishable balls into k distinguishable bins is:

$$\binom{n+k-1}{k-1}.$$

With each bin having ≥ 1 ball: $\binom{n-1}{k-1}$.

Intuition. Place $k-1$ dividers among $n+k-1$ positions.

CP Utility: Counting integer solutions to $x_1 + x_2 + \cdots + x_k = n$ with $x_i \geq 0$.

2.3.1 Python Implementation

```python
def stars_bars(n, k, mod=10**9+7):
    """Number of ways: x1+...+xk = n, xi >= 0."""
    return comb_mod(n + k - 1, k - 1, mod)

def stars_bars_positive(n, k, mod=10**9+7):
    """Number of ways: x1+...+xk = n, xi >= 1."""
    if n < k: return 0
    return comb_mod(n - 1, k - 1, mod)

def comb_mod(n, k, mod=10**9+7):
    if k < 0 or k > n: return 0
    # Use small pascal for small n, or factorial method
    num, den = 1, 1
    k = min(k, n - k)
    for i in range(k):
        num = num * (n - i) % mod
        den = den * (i + 1) % mod
    return num * pow(den, mod-2, mod) % mod

print(stars_bars(5, 3))    # C(7,2) = 21
```

Listing 14: Stars and Bars – non-negative and positive integer solutions

2.4 Inclusion–Exclusion Principle

Theorem 2.3 (Inclusion–Exclusion)**.** For finite sets $A_1, \ldots, A_n$:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \cdots + (-1)^{n+1}|A_1 \cap \cdots \cap A_n|.$$

Intuition. Add sets, subtract over-counting of pairwise intersections, add back triple intersections, etc.

CP Utility: Counting elements satisfying at least one of several conditions; complementary counting; Möbius-style problems.

2.4.1 Python Implementation

```python
def count_multiples_iec(N, primes):
    """Count integers in [1,N] divisible by at least one prime."""
    n = len(primes)
    total = 0
    for mask in range(1, 1 << n):
        prod = 1
        bits = bin(mask).count('1')
        for i in range(n):
            if mask & (1 << i):
                prod *= primes[i]
                if prod > N: break
        else:
            cnt = N // prod
            if bits % 2 == 1:
                total += cnt
            else:
                total -= cnt
    return total

print(count_multiples_iec(30, [2, 3, 5]))  # 22
```

Listing 15:]Inclusion-Exclusion – count multiples of primes in [1,N]

Complexity. $\mathcal{O}(2^n)$ for n sets.

2.5 Pigeonhole Principle

Theorem 2.4 (Pigeonhole Principle)**.** If $n > k$ objects are distributed into k containers, at least one container has ≥ 2 objects. More generally, some container has $\geq \lceil n/k \rceil$ objects.

Intuition. Averaging argument: if all bins had $< \lceil n/k \rceil$, the total would be less than n.

CP Utility: Proving existence; bounding maximum; guarantee-type arguments; hash collisions.

Example. In any sequence of $n^2 + 1$ distinct integers, there is a monotone subsequence of length $n + 1$ (Dilworth / Erdős–Szekeres).

2.6 Catalan Numbers

Theorem 2.5 (Catalan Numbers)**.** The n-th Catalan number is:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!\,n!}.$$

$C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14, \ldots$ Recurrence: $C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$.

Where to use. Balanced parentheses, BST shapes, triangulations, non-crossing partitions, mountain ranges (Dyck paths).

2.6.1 Python Implementation

```python
def catalan(n, mod=10**9+7):
    """C_n = C(2n,n)/(n+1) mod p."""
    # Precompute factorials
    N = 2 * n + 2
    fact = [1] * (N + 1)
    for i in range(1, N + 1):
        fact[i] = fact[i-1] * i % mod
    inv_fact = [1] * (N + 1)
    inv_fact[N] = pow(fact[N], mod-2, mod)
    for i in range(N-1, -1, -1):
        inv_fact[i] = inv_fact[i+1] * (i+1) % mod
```

```python
    return fact[2*n] * inv_fact[n] % mod * inv_fact[n+1] % mod

# First 10 Catalan numbers
for i in range(10):
    print(i, catalan(i))
# 0:1, 1:1, 2:2, 3:5, 4:14, 5:42, 6:132, 7:429, 8:1430, 9:4862
```

Listing 16: Catalan Numbers mod p

Complexity. $\mathcal{O}(n)$ per value.

2.7 Sperner's Theorem

Theorem 2.6 (Sperner's Theorem)**.** The maximum size of an antichain (a family of sets no one of which contains another) in the power set of an n-element set is $\binom{n}{\lfloor n/2 \rfloor}$.

Intuition. Among all layers of the Boolean lattice $\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}$, the middle layer $\binom{n}{\lfloor n/2 \rfloor}$ is the largest, and no two sets of equal size can contain one another.

Mathematical formulation (LYM inequality).

$$\sum_{A \in \mathcal{F}} \frac{1}{\binom{n}{|A|}} \leq 1 \quad \Longrightarrow \quad |\mathcal{F}| \leq \binom{n}{\lfloor n/2 \rfloor}.$$

CP Utility: Bounding families of subsets; proving impossibility of certain constructions; extremal set-system arguments.

Where to use. Problems asking for maximum subset families with no containment; proof arguments in combinatorial optimization.

Example Problem. Given n elements, find the maximum number of subsets such that no subset is contained in another.

2.7.1 Python Implementation

```python
from math import comb

def max_antichain_size(n):
    """Maximum antichain in power set of {1,...,n}."""
    return comb(n, n // 2)

def is_antichain(family):
    """Check if a family of sets forms an antichain."""
    fam = [set(s) for s in family]
    for i in range(len(fam)):
        for j in range(len(fam)):
            if i != j and fam[i] <= fam[j]:
                return False
    return True

# For n=4, max antichain = C(4,2) = 6 sets
print(max_antichain_size(4))  # 6
family = [{0,1},{0,2},{0,3},{1,2},{1,3},{2,3}]
print(is_antichain(family))   # True
```

Listing 17: Sperner's Theorem – max antichain and verification

Complexity. Computing $\binom{n}{\lfloor n/2 \rfloor}$: $\mathcal{O}(n)$; verifying antichain: $\mathcal{O}\big(|\mathcal{F}|^2 \cdot n\big)$.

2.8 Derangements

Definition 2.7. A derangement of n elements is a permutation with no fixed points. The count is:

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} \approx \frac{n!}{e}.$$

Recurrence: $D_n = (n-1)(D_{n-1} + D_{n-2})$, $D_0 = 1, D_1 = 0$.

2.8.1 Python Implementation

```python
def derangements(N, mod=10**9+7):
    """Returns D[0..N] where D[n] = number of derangements of n."""
    D = [0] * (N + 1)
    D[0] = 1
    if N >= 1: D[1] = 0
    for n in range(2, N + 1):
        D[n] = (n - 1) * (D[n-1] + D[n-2]) % mod
    return D

D = derangements(10)
print(D[4])  # 9
print(D[5])  # 44
```

Listing 18: Derangements mod p

2.9 Combinations with Repetition

Theorem 2.8. The number of multisets of size k from an n-element set is:

$$\binom{n+k-1}{k} = \binom{n+k-1}{n-1}.$$

Intuition. Equivalent to Stars and Bars: choose k elements with repetition from n types.

2.9.1 Python Implementation

```python
def comb_rep(n, k, mod=10**9+7):
    """Multisets: choose k from n types with repetition."""
    return comb_mod(n + k - 1, k, mod)

def comb_mod(n, k, mod=10**9+7):
    if k < 0 or k > n: return 0
    k = min(k, n-k)
    num = den = 1
    for i in range(k):
        num = num * (n - i) % mod
        den = den * (i + 1) % mod
    return num * pow(den, mod-2, mod) % mod

print(comb_rep(3, 2))  # 6 (aa,ab,ac,bb,bc,cc)
```

Listing 19: Combinations with repetition

3 Graph Theory

3.1 BFS – Breadth-First Search

Algorithm 3.1 (BFS). Explore vertices level by level from a source. Uses a queue. Finds shortest paths in unweighted graphs.

Where to use. Shortest path in unweighted graph; 0-1 BFS; level-order traversal; bipartite check.

Typical CP scenarios. Grid shortest path (4/8-directional); multi-source BFS (spread from multiple sources simultaneously); BFS on states (e.g., Rubik's cube, game states).

Example Problem. Given a grid with `#` walls and `.` paths, find the shortest path from top-left to bottom-right.

3.1.1 Python Implementation

```python
from collections import deque

def bfs(graph, src, n):
    """graph: adjacency list. Returns dist[] from src."""
    dist = [-1] * n
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

# 0-1 BFS for graphs with edge weights 0 or 1
def bfs_01(graph, src, n):
    dist = [float('inf')] * n
    dist[src] = 0
    dq = deque([src])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist
```

Listing 20: BFS – shortest paths

Complexity. $\mathcal{O}(V + E)$.

> **Note / Pitfall:** For multi-source BFS, push all sources into the queue initially with distance 0. For states involving $\{\text{position}, \text{keys collected}\}$, BFS on the augmented state space.

3.2 DFS – Depth-First Search

Algorithm 3.2 (DFS). Explore as deep as possible before backtracking. Can be recursive or iterative. Produces DFS tree, discovery/finish times.

Where to use. Connected components; cycle detection; topological sort; SCC; bridges/articulation points; backtracking.

Typical CP scenarios. Finding connected components; detecting back edges (cycles); tree diameter via two DFS calls; flood fill.

Example Problem. Given an undirected graph, count the number of connected components.

3.2.1 Python Implementation

```python
def dfs(graph, n):
    visited = [False] * n
    order = []
    def dfs_visit(u):
        stack = [(u, 0)]
        while stack:
            v, idx = stack.pop()
            if idx == 0:
                if visited[v]: continue
                visited[v] = True
                order.append(v)
                stack.append((v, 1))
                for w in reversed(graph[v]):
                    if not visited[w]:
                        stack.append((w, 0))
    for i in range(n):
        if not visited[i]:
            dfs_visit(i)
    return order
```

Listing 21: DFS – iterative with timestamps

Complexity. $\mathcal{O}(V + E)$.

Note / Pitfall: Python's default recursion limit is 1000. For DFS on large graphs, always use iterative DFS or raise the limit with `sys.setrecursionlimit()` and use threading for larger stack.

3.3 Dijkstra's Algorithm

Algorithm 3.3 (Dijkstra)**.** Single-source shortest paths in graphs with non-negative edge weights. Uses a priority queue (min-heap).

Where to use. Weighted graphs with non-negative edges; network routing; shortest path in state spaces (BFS with costs).

Typical CP scenarios. Road networks; weighted grids; shortest path with constraints (multi-state Dijkstra).

Example Problem. Given n cities and m weighted roads, find the shortest distance from city 1 to city n.

$$d[v] = \min_{(u,v)\in E} (d[u] + w(u, v))$$

3.3.1 Python Implementation

```python
import heapq

def dijkstra(graph, src, n):
    """graph[u] = [(v, w), ...]. Returns dist[] from src."""
    dist = [float('inf')] * n
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
```

```
                heapq.heappush(heap, (dist[v], v))
    return dist

# Example
n = 5
g = [[] for _ in range(n)]
g[0].append((1, 2)); g[0].append((2, 4))
g[1].append((2, 1)); g[1].append((3, 7))
g[2].append((4, 3)); g[3].append((4, 1))
print(dijkstra(g, 0, n))  # [0, 2, 3, 9, 6]
```

Listing 22: Dijkstra with heapq

Complexity. $\mathcal{O}((V+E)\log V)$ with binary heap.

Note / Pitfall: Dijkstra fails with negative edges. Use Bellman-Ford instead.

3.4 Bellman-Ford Algorithm

Algorithm 3.4 (Bellman-Ford)**.** Relax all edges $V-1$ times. Detects negative cycles in a V-th relaxation pass.

Where to use. Graphs with negative edge weights; detecting negative cycles; when Dijkstra is inapplicable.

Typical CP scenarios. Currency arbitrage (negative cycle = profit); shortest path with negative edges; SPFA optimization.

Example Problem. Given a currency exchange graph, determine if there is an arbitrage opportunity (negative cycle).

3.4.1 Python Implementation

```
def bellman_ford(edges, src, n):
    """edges = [(u,v,w)]. Returns (dist, has_neg_cycle)."""
    dist = [float('inf')] * n
    dist[src] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # Check for negative cycle
    neg_cycle = False
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            neg_cycle = True
            break
    return dist, neg_cycle
```

Listing 23: Bellman-Ford with negative cycle detection

Complexity. $\mathcal{O}(VE)$.

Note / Pitfall: SPFA (Shortest Path Faster Algorithm) is Bellman-Ford with a queue – often much faster in practice but $\mathcal{O}(VE)$ worst case. Avoid on adversarial test cases.

3.5 Floyd-Warshall Algorithm

Algorithm 3.5 (Floyd-Warshall)**.** All-pairs shortest paths in $\mathcal{O}(V^3)$.

$$d[i][j] = \min(d[i][j],\ d[i][k] + d[k][j]) \quad \forall k$$

Where to use. Small dense graphs ($V \le 500$); transitive closure; detecting negative cycles ($d[i][i] < 0$).

Typical CP scenarios. All-pairs shortest paths; reachability; minimum cycle through a node.

Example Problem. Given $n \le 400$ cities with direct flights, find the shortest round trip through any city.

3.5.1 Python Implementation

```python
INF = float('inf')

def floyd_warshall(dist):
    """dist[i][j] = initial edge weight (INF if no edge), in-place."""
    n = len(dist)
    for k in range(n):
        for i in range(n):
            if dist[i][k] == INF: continue
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    # Negative cycle: dist[i][i] < 0
    return dist

# Usage
n = 4
d = [[INF]*n for _ in range(n)]
for i in range(n): d[i][i] = 0
d[0][1]=3; d[0][3]=7; d[1][0]=8; d[1][2]=2
d[2][0]=5; d[2][3]=1; d[3][0]=2
floyd_warshall(d)
```

Listing 24: Floyd-Warshall all-pairs shortest paths

Complexity. $\mathcal{O}(V^3)$. Best for dense graphs with $V \leq 500$.

3.6 Topological Sort

Algorithm 3.6 (Topological Sort (Kahn's + DFS))**.** Linear ordering of vertices in a DAG such that for every edge (u, v), u comes before v.

3.6.1 Python Implementation

```python
from collections import deque

def topo_kahn(graph, n):
    """Kahn's algorithm. Returns [] if cycle exists."""
    indegree = [0] * n
    for u in range(n):
        for v in graph[u]:
            indegree[v] += 1
    q = deque(i for i in range(n) if indegree[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                q.append(v)
    return order if len(order) == n else []  # Empty = cycle

def topo_dfs(graph, n):
    """DFS-based topological sort."""
    visited = [0] * n  # 0=unvisited, 1=in stack, 2=done
    order = []
    def dfs(u):
        visited[u] = 1
        for v in graph[u]:
            if visited[v] == 1: return False  # Cycle
            if visited[v] == 0:
                if not dfs(v): return False
        visited[u] = 2
```

```
        order.append(u)
        return True
    for i in range(n):
        if visited[i] == 0:
            if not dfs(i): return []
    return order[::-1]
```

Listing 25: Topological Sort – Kahn's BFS and DFS

Complexity. $\mathcal{O}(V + E)$.

3.7 Kruskal's Algorithm (MST)

Algorithm 3.7 (Kruskal)**.** Sort edges by weight; greedily add the minimum weight edge that doesn't form a cycle (use DSU).

3.7.1 Python Implementation

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b: return False
        if self.rank[a] < self.rank[b]: a, b = b, a
        self.p[b] = a
        if self.rank[a] == self.rank[b]: self.rank[a] += 1
        return True

def kruskal(n, edges):
    """edges = [(w,u,v)]. Returns (mst_weight, mst_edges)."""
    edges.sort()
    dsu = DSU(n)
    mst_w = 0
    mst_edges = []
    for w, u, v in edges:
        if dsu.union(u, v):
            mst_w += w
            mst_edges.append((u, v, w))
            if len(mst_edges) == n - 1:
                break
    return mst_w, mst_edges
```

Listing 26: Kruskal's MST with DSU

Complexity. $\mathcal{O}(E \log E)$.

3.8 Prim's Algorithm (MST)

3.8.1 Python Implementation

```
import heapq

def prim(graph, n, src=0):
    """graph[u] = [(v,w)]. Returns MST weight."""
    visited = [False] * n
```

```python
    heap = [(0, src)]
    total = 0
    while heap:
        w, u = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        total += w
        for v, wt in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (wt, v))
    return total
```

Listing 27: Prim's MST with priority queue

Complexity. $\mathcal{O}((V + E) \log V)$.

3.9 Union-Find (DSU)

Already shown in Kruskal; additional features below.

3.9.1 Python Implementation

```python
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n
        self.n_components = n

    def find(self, x):
        r = x
        while self.p[r] != r: r = self.p[r]
        while self.p[x] != r:
            self.p[x], x = r, self.p[x]
        return r

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b: return False
        if self.size[a] < self.size[b]: a, b = b, a
        self.p[b] = a
        self.size[a] += self.size[b]
        self.n_components -= 1
        return True

    def same(self, a, b): return self.find(a) == self.find(b)
    def comp_size(self, a): return self.size[self.find(a)]
```

Listing 28: DSU with size tracking and rollback

Complexity. Near $\mathcal{O}(1)$ amortized (inverse Ackermann).

3.10 Tarjan's SCC

Algorithm 3.8 (Tarjan SCC)**.** DFS-based; maintains a stack and discovery/low-link times. SCCs are popped when root is identified.

3.10.1 Python Implementation

```python
import sys
sys.setrecursionlimit(300000)

def tarjan_scc(graph, n):
    disc = [-1] * n
    low = [0] * n
```

```python
    on_stack = [False] * n
    stack = []
    sccs = []
    timer = [0]

    def dfs(u):
        disc[u] = low[u] = timer[0]
        timer[0] += 1
        stack.append(u)
        on_stack[u] = True
        for v in graph[u]:
            if disc[v] == -1:
                dfs(v)
                low[u] = min(low[u], low[v])
            elif on_stack[v]:
                low[u] = min(low[u], disc[v])
        if low[u] == disc[u]:
            scc = []
            while True:
                v = stack.pop()
                on_stack[v] = False
                scc.append(v)
                if v == u: break
            sccs.append(scc)

    for i in range(n):
        if disc[i] == -1:
            dfs(i)
    return sccs
```

Listing 29: Tarjan's Strongly Connected Components

Complexity. $\mathcal{O}(V + E)$.

Note / Pitfall: This recursive implementation hits Python's recursion limit for $n > 10^4$. For large graphs, use Kosaraju's (iterative) or implement an iterative Tarjan using an explicit stack.

3.11 Kosaraju's SCC

3.11.1 Python Implementation

```python
def kosaraju(graph, rgraph, n):
    """graph and rgraph: adjacency lists."""
    visited = [False] * n
    order = []

    def dfs1(u):
        stack = [(u, 0)]
        while stack:
            v, state = stack.pop()
            if state == 0:
                if visited[v]: continue
                visited[v] = True
                stack.append((v, 1))
                for w in graph[v]:
                    if not visited[w]:
                        stack.append((w, 0))
            else:
                order.append(v)

    for i in range(n):
        if not visited[i]:
            dfs1(i)

    comp = [-1] * n
```

```python
    def dfs2(u, c):
        stack = [u]
        while stack:
            v = stack.pop()
            if comp[v] != -1: continue
            comp[v] = c
            for w in rgraph[v]:
                if comp[w] == -1:
                    stack.append(w)

    c = 0
    for u in reversed(order):
        if comp[u] == -1:
            dfs2(u, c)
            c += 1
    return comp, c
```

Listing 30: Kosaraju's SCC

Complexity. $\mathcal{O}(V + E)$.

3.12 Bridges and Articulation Points

3.12.1 Python Implementation

```python
def find_bridges_ap(graph, n):
    disc = [-1] * n
    low = [0] * n
    parent = [-1] * n
    bridges = []
    ap = [False] * n
    timer = [0]

    def dfs(u):
        disc[u] = low[u] = timer[0]
        timer[0] += 1
        child_count = 0
        for v in graph[u]:
            if disc[v] == -1:
                child_count += 1
                parent[v] = u
                dfs(v)
                low[u] = min(low[u], low[v])
                if low[v] > disc[u]:
                    bridges.append((u, v))
                if parent[u] == -1 and child_count > 1:
                    ap[u] = True
                if parent[u] != -1 and low[v] >= disc[u]:
                    ap[u] = True
            elif v != parent[u]:
                low[u] = min(low[u], disc[v])

    for i in range(n):
        if disc[i] == -1:
            dfs(i)
    return bridges, [i for i in range(n) if ap[i]]
```

Listing 31: Bridges and Articulation Points via DFS

Complexity. $\mathcal{O}(V + E)$.

Note / Pitfall: This recursive implementation may exceed Python's stack limit for $n > 10^4$. Use `sys.setrecursionlimit` or rewrite iteratively. For multi-edge graphs, track edge index rather than `v != parent[u]` to handle duplicate edges correctly.

3.13 LCA with Binary Lifting

3.13.1 Python Implementation

```python
import math

class LCA:
    def __init__(self, tree, root, n):
        LOG = max(1, math.floor(math.log2(n)) + 1) if n > 1 else 1
        self.LOG = LOG
        self.depth = [0] * n
        self.up = [[-1] * n for _ in range(LOG)]
        # BFS to set depth and up[0]
        from collections import deque
        visited = [False] * n
        q = deque([root])
        visited[root] = True
        self.up[0][root] = root
        while q:
            u = q.popleft()
            for v in tree[u]:
                if not visited[v]:
                    visited[v] = True
                    self.depth[v] = self.depth[u] + 1
                    self.up[0][v] = u
                    q.append(v)
        for k in range(1, LOG):
            for v in range(n):
                self.up[k][v] = self.up[k-1][self.up[k-1][v]]

    def lca(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u
        diff = self.depth[u] - self.depth[v]
        for k in range(self.LOG):
            if (diff >> k) & 1:
                u = self.up[k][u]
        if u == v: return u
        for k in range(self.LOG - 1, -1, -1):
            if self.up[k][u] != self.up[k][v]:
                u = self.up[k][u]
                v = self.up[k][v]
        return self.up[0][u]
```

Listing 32: LCA using Binary Lifting

Complexity. $\mathcal{O}(N \log N)$ build; $\mathcal{O}(\log N)$ query.

3.14 Euler Tour

3.14.1 Python Implementation

```python
def euler_tour(tree, root, n):
    tin = [0] * n
    tout = [0] * n
    order = []
    timer = [0]
    stack = [(root, -1, False)]
    while stack:
        u, par, leaving = stack.pop()
        if leaving:
            tout[u] = timer[0]
            timer[0] += 1
        else:
            tin[u] = timer[0]
            timer[0] += 1
```

```
            order.append(u)
            stack.append((u, par, True))
            for v in tree[u]:
                if v != par:
                    stack.append((v, u, False))
    return tin, tout, order

# Node v is in subtree of u iff tin[u] <= tin[v] < tout[u]
```

Listing 33: Euler Tour – in/out times for subtree queries

Complexity. $\mathcal{O}(N)$.

3.15 Maximum Flow – Dinic's Algorithm

3.15.1 Python Implementation

```
from collections import deque

class Dinic:
    def __init__(self, n):
        self.n = n
        self.graph = [[] for _ in range(n)]

    def add_edge(self, u, v, cap):
        self.graph[u].append([v, cap, len(self.graph[v])])
        self.graph[v].append([u, 0, len(self.graph[u])-1])

    def bfs(self, s, t):
        self.level = [-1] * self.n
        self.level[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v, cap, _ in self.graph[u]:
                if cap > 0 and self.level[v] < 0:
                    self.level[v] = self.level[u] + 1
                    q.append(v)
        return self.level[t] >= 0

    def dfs(self, u, t, f):
        if u == t: return f
        while self.iter[u] < len(self.graph[u]):
            v, cap, rev = self.graph[u][self.iter[u]]
            if cap > 0 and self.level[v] == self.level[u] + 1:
                d = self.dfs(v, t, min(f, cap))
                if d > 0:
                    self.graph[u][self.iter[u]][1] -= d
                    self.graph[v][rev][1] += d
                    return d
            self.iter[u] += 1
        return 0

    def max_flow(self, s, t):
        flow = 0
        while self.bfs(s, t):
            self.iter = [0] * self.n
            while True:
                f = self.dfs(s, t, float('inf'))
                if f == 0: break
                flow += f
        return flow
```

Listing 34: Dinic's Maximum Flow

Complexity. $\mathcal{O}(V^2E)$; $\mathcal{O}\left(E\sqrt{V}\right)$ for unit graphs.

3.16 Bipartite Matching (Hopcroft-Karp)

3.16.1 Python Implementation

```python
from collections import deque
INF = float('inf')

def hopcroft_karp(adj, n, m):
    """adj[u] = list of v for left u, right v. Returns matching size."""
    match_l = [-1] * n
    match_r = [-1] * m
    dist = [0] * n

    def bfs():
        queue = deque()
        for u in range(n):
            if match_l[u] == -1:
                dist[u] = 0
                queue.append(u)
            else:
                dist[u] = INF
        found = False
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                w = match_r[v]
                if w == -1:
                    found = True
                elif dist[w] == INF:
                    dist[w] = dist[u] + 1
                    queue.append(w)
        return found

    def dfs(u):
        for v in adj[u]:
            w = match_r[v]
            if w == -1 or (dist[w] == dist[u] + 1 and dfs(w)):
                match_l[u] = v
                match_r[v] = u
                return True
        dist[u] = INF
        return False

    matching = 0
    while bfs():
        for u in range(n):
            if match_l[u] == -1 and dfs(u):
                matching += 1
    return matching, match_l, match_r
```

Listing 35: Hopcroft-Karp maximum bipartite matching

Complexity. $\mathcal{O}\left(E\sqrt{V}\right)$.

3.17 Minimum Cut

Theorem 3.9 (Max-Flow Min-Cut Theorem)**.** In a flow network, the maximum flow equals the minimum cut capacity.

Finding the min-cut. After running max flow, do BFS/DFS from source in the residual graph. Edges from reachable to non-reachable vertices form the min cut.

Example Problem. Given a network of pipes with capacities, find the minimum total capacity of pipes to cut so no water flows from source to sink.

3.17.1 Python Implementation

```python
from collections import deque

def find_min_cut(dinic, s):
    """After dinic.max_flow(s, t), find reachable set from s
    in residual graph. Cut = edges from reachable to rest."""
    reachable = set()
    q = deque([s])
    reachable.add(s)
    while q:
        u = q.popleft()
        for v, cap, _ in dinic.graph[u]:
            if cap > 0 and v not in reachable:
                reachable.add(v)
                q.append(v)
    # Cut edges: u reachable, v not, original cap > 0
    cut = []
    for u in reachable:
        for v, cap, rev in dinic.graph[u]:
            if v not in reachable and cap == 0:
                orig = dinic.graph[v][rev][1]
                if orig > 0:
                    cut.append((u, v, orig))
    return reachable, cut

# Usage: flow = dinic.max_flow(s, t)
# reachable, cut_edges = find_min_cut(dinic, s)
# sum of cut capacities == flow
```

Listing 36: Minimum Cut – extract cut edges after max flow

Complexity. BFS is $\mathcal{O}(V + E)$ on top of the max-flow computation.

Note / Pitfall: For minimum vertex cut, split each vertex v into v_{in} and v_{out} with an edge of capacity 1, then run max flow.

3.18 Bipartite Graph Theorem

Theorem 3.10. A graph is bipartite if and only if it contains no odd-length cycle.

Test via 2-coloring BFS/DFS.

3.18.1 Python Implementation

```python
from collections import deque

def is_bipartite(graph, n):
    color = [-1] * n
    for start in range(n):
        if color[start] != -1: continue
        color[start] = 0
        q = deque([start])
        while q:
            u = q.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    q.append(v)
                elif color[v] == color[u]:
                    return False, color
```

```
    return True, color
```

Listing 37: Bipartite check via 2-coloring

4 Dynamic Programming

4.1 0/1 Knapsack

Algorithm 4.1 (0/1 Knapsack)**.** Given items with weights w_i and values v_i, maximize $\sum v_i x_i$ subject to $\sum w_i x_i \leq W$, $x_i \in \{0, 1\}$.

$$dp[j] = \max(dp[j],\ dp[j - w_i] + v_i) \quad \text{(iterate } j \text{ backwards)}$$

4.1.1 Python Implementation

```python
def knapsack_01(items, W):
    """items = [(weight, value)]. Returns max value."""
    dp = [0] * (W + 1)
    for w, v in items:
        for j in range(W, w - 1, -1):
            dp[j] = max(dp[j], dp[j - w] + v)
    return dp[W]

items = [(2,3),(3,4),(4,5),(5,6)]
print(knapsack_01(items, 8))  # 10
```

Listing 38: 0/1 Knapsack

Complexity. $\mathcal{O}(nW)$.

4.2 Unbounded Knapsack

4.2.1 Python Implementation

```python
def knapsack_unbounded(items, W):
    dp = [0] * (W + 1)
    for j in range(1, W + 1):
        for w, v in items:
            if w <= j:
                dp[j] = max(dp[j], dp[j - w] + v)
    return dp[W]
```

Listing 39: Unbounded Knapsack

Key difference from 0/1. Iterate j *forward* (each item usable multiple times).

4.3 Coin Change

4.3.1 Python Implementation

```python
def coin_change_min(coins, amount):
    """Min number of coins to make amount. -1 if impossible."""
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for c in coins:
        for j in range(c, amount + 1):
            dp[j] = min(dp[j], dp[j - c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

def coin_change_ways(coins, amount, mod=10**9+7):
    """Count distinct ways to make amount."""
    dp = [0] * (amount + 1)
    dp[0] = 1
    for c in coins:
        for j in range(c, amount + 1):
            dp[j] = (dp[j] + dp[j - c]) % mod
    return dp[amount]

```

```python
print(coin_change_min([1,5,6,9], 11))  # 2 (5+6)
print(coin_change_ways([1,2,5], 5))    # 4
```

Listing 40: Coin Change – min coins and count ways

4.4 LIS – Longest Increasing Subsequence $\mathcal{O}(n \log n)$

Algorithm 4.2 (LIS in $\mathcal{O}(n \log n)$)**.** Maintain array `tails` where `tails`$[i]$ is the smallest tail of all increasing subsequences of length $i+1$. Use binary search to update.

4.4.1 Python Implementation

```python
import bisect

def lis(arr):
    """Returns length of LIS."""
    tails = []
    for x in arr:
        pos = bisect.bisect_left(tails, x)
        if pos == len(tails):
            tails.append(x)
        else:
            tails[pos] = x
    return len(tails)

def lis_sequence(arr):
    """Returns one actual LIS (not just the length)."""
    n = len(arr)
    tails = []        # tails[i] = smallest tail of IS length i+1
    tail_idx = []      # index in arr for tails[i]
    prev_idx = [-1] * n
    for i, x in enumerate(arr):
        p = bisect.bisect_left(tails, x)
        if p == len(tails):
            tails.append(x)
            tail_idx.append(i)
        else:
            tails[p] = x
            tail_idx[p] = i
        prev_idx[i] = tail_idx[p - 1] if p > 0 else -1
    # Reconstruct backwards from last element
    result = []
    idx = tail_idx[-1]
    while idx != -1:
        result.append(arr[idx])
        idx = prev_idx[idx]
    return result[::-1]

print(lis([10,9,2,5,3,7,101,18]))  # 4
print(lis_sequence([10,9,2,5,3,7,101,18]))  # [2,3,7,18]
```

Listing 41: LIS O(n log n) with patience sorting

Complexity. $\mathcal{O}(n \log n)$.

4.5 LCS – Longest Common Subsequence

4.5.1 Python Implementation

```python
def lcs(s, t):
    n, m = len(s), len(t)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, m+1):
```

```python
            if s[i-1] == t[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    # Reconstruct
    seq = []
    i, j = n, m
    while i > 0 and j > 0:
        if s[i-1] == t[j-1]:
            seq.append(s[i-1])
            i -= 1; j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            i -= 1
        else:
            j -= 1
    return dp[n][m], ''.join(reversed(seq))

print(lcs("ABCBDAB","BDCAB"))  # (4, 'BCAB')
```

Listing 42: LCS – O(nm) DP

Complexity. $\mathcal{O}(nm)$ time and space (optimizable to $\mathcal{O}(m)$ space).

4.6 Interval DP

Algorithm 4.3 (Interval DP)**.** DP over contiguous intervals $[l, r]$, expanding from small to large. Iterate over interval length, then left endpoint, then split point k:

$$dp[l][r] = \min_{l \le k < r} \big(dp[l][k] + dp[k{+}1][r] + \mathrm{cost}(l, r)\big).$$

Where to use. Matrix chain multiplication; optimal BST; merging stones; bursting balloons; palindrome partitioning.

Typical CP scenarios. Merging adjacent elements with additive costs; splitting or grouping a sequence optimally.

Example Problem. Given n piles of stones in a row, repeatedly merge two adjacent piles (cost = sum of the two). Minimize total cost.

4.6.1 Python Implementation

```python
def merge_stones(stones):
    """Minimum cost to merge all adjacent piles into one."""
    n = len(stones)
    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i+1] = prefix[i] + stones[i]
    INF = float('inf')
    dp = [[0]*n for _ in range(n)]
    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            dp[l][r] = INF
            cost = prefix[r+1] - prefix[l]
            for k in range(l, r):
                dp[l][r] = min(dp[l][r],
                               dp[l][k] + dp[k+1][r] + cost)
    return dp[0][n-1]

def matrix_chain(dims):
    """dims = [d0,...,dn]. Matrix i is dims[i]*dims[i+1].
    Minimize scalar multiplications."""
    n = len(dims) - 1
    dp = [[0]*n for _ in range(n)]
    for length in range(2, n + 1):
```

```python
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j]
                               + dims[i]*dims[k+1]*dims[j+1])
    return dp[0][n-1]

print(merge_stones([3,4,5,2]))  # 33
print(matrix_chain([10,30,5,60]))  # 4500
```

Listing 43: Interval DP – merge stones and matrix chain

Complexity. $\mathcal{O}(n^3)$ time; $\mathcal{O}(n^2)$ space.

Note / Pitfall: For $n > 500$, check if Knuth's optimization applies (quadrangle inequality) to reduce to $\mathcal{O}(n^2)$.

4.7 Bitmask DP

4.7.1 Python Implementation

```python
def tsp(dist, n):
    """Minimum Hamiltonian cycle. dist[i][j] = cost."""
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0  # Start at node 0, visited = {0}
    for mask in range(1 << n):
        for u in range(n):
            if not (mask >> u & 1): continue
            if dp[mask][u] == INF: continue
            for v in range(n):
                if mask >> v & 1: continue
                nmask = mask | (1 << v)
                dp[nmask][v] = min(dp[nmask][v], dp[mask][u] + dist[u][v])
    full = (1 << n) - 1
    ans = min(dp[full][u] + dist[u][0] for u in range(n))
    return ans
```

Listing 44: TSP via Bitmask DP

Complexity. $\mathcal{O}(2^n \cdot n^2)$.

4.8 Digit DP

4.8.1 Python Implementation

```python
from functools import lru_cache

def digit_dp(N, K):
    """Count integers in [1, N] with digit sum divisible by K."""
    digits = list(map(int, str(N)))
    n = len(digits)

    @lru_cache(maxsize=None)
    def dp(pos, rem, tight, started):
        if pos == n:
            return 1 if (rem == 0 and started) else 0
        limit = digits[pos] if tight else 9
        res = 0
        for d in range(0, limit + 1):
            new_rem = (rem + d) % K if (started or d > 0) else 0
            new_started = started or (d > 0)
            res += dp(pos+1, new_rem, tight and d==limit, new_started)
```

```python
        return res

    return dp(0, 0, True, False)

print(digit_dp(100, 3))  # count in [1,100] with digit sum % 3 == 0
```

Listing 45: Digit DP – count numbers with digit sum divisible by k

Complexity. $\mathcal{O}(\text{digits} \times K \times 10)$.

4.9 Tree DP

4.9.1 Python Implementation

```python
def tree_max_independent_set(tree, n, root=0):
    """dp[v][0] = max set not including v, dp[v][1] = including v."""
    dp = [[0, 1] for _ in range(n)]  # [exclude, include]
    parent = [-1] * n
    order = []
    stack = [root]
    visited = [False] * n
    visited[root] = True
    while stack:
        u = stack.pop()
        order.append(u)
        for v in tree[u]:
            if not visited[v]:
                visited[v] = True
                parent[v] = u
                stack.append(v)
    for u in reversed(order):
        for v in tree[u]:
            if parent[v] == u:
                dp[u][0] += max(dp[v][0], dp[v][1])
                dp[u][1] += dp[v][0]
    return max(dp[root][0], dp[root][1])
```

Listing 46: Tree DP – maximum independent set on tree

4.10 DP with Prefix Sums

Pattern. When $dp[i] = \sum_{j<i} f(dp[j])$, precompute prefix sums of dp to answer range queries in $\mathcal{O}(1)$.

4.10.1 Python Implementation

```python
def count_sequences(n, mod=10**9+7):
    """Example: dp[i] = sum of dp[j] for all valid j < i."""
    dp = [0] * (n + 1)
    prefix = [0] * (n + 2)
    dp[0] = 1
    prefix[1] = 1
    for i in range(1, n + 1):
        # Example: valid j in [i//2, i-1]
        lo, hi = i // 2, i - 1
        dp[i] = (prefix[hi + 1] - prefix[lo]) % mod
        prefix[i + 1] = (prefix[i] + dp[i]) % mod
    return dp[n]
```

Listing 47: DP with prefix sums – count sequences

4.11 Meet-in-the-Middle DP

Idea. Split the problem into two halves of size $n/2$. Solve each independently ($\mathcal{O}(2^{n/2})$ each), then combine. Reduces $\mathcal{O}(2^n)$ to $\mathcal{O}(2^{n/2})$.

4.11.1 Python Implementation

```python
def meet_in_middle(arr, target):
    """Check if any subset sums to target."""
    n = len(arr)
    half = n // 2
    left, right = arr[:half], arr[half:]

    def all_subset_sums(a):
        sums = set()
        for mask in range(1 << len(a)):
            s = sum(a[i] for i in range(len(a)) if mask >> i & 1)
            sums.add(s)
        return sums

    left_sums = all_subset_sums(left)
    right_sums = all_subset_sums(right)
    for s in left_sums:
        if target - s in right_sums:
            return True
    return False
```

Listing 48: Meet-in-the-Middle – subset sum

Complexity. $\mathcal{O}(2^{n/2} \log n)$.

4.12 State Compression DP

Concept. Represent sets/subsets as bitmasks (integers). Enumerate all 2^n subsets efficiently.

```python
# Iterate all subsets of mask (including mask itself and 0)
def subsets(mask):
    s = mask
    while s > 0:
        yield s
        s = (s - 1) & mask
    yield 0

# Sum over subsets (SOS DP)
def sos_dp(f, n):
    """f[mask] = sum over subsets after SOS DP."""
    for i in range(n):
        for mask in range(1 << n):
            if mask >> i & 1:
                f[mask] += f[mask ^ (1 << i)]
    return f
```

Listing 49: Subset enumeration – iterate all subsets of a mask

4.13 DP Optimizations

- **Divide and Conquer DP:** When $opt(i, j)$ is monotone. Reduces $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$.
- **Knuth's Optimization:** For interval DP when quadrangle inequality holds. Reduces $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$.
- **Convex Hull Trick:** When $dp[i] = \min_j(f(j) + x_i \cdot g(j))$ and x_i is monotone.

4.13.1 Python Implementation

```python
class LiChaoTree:
```

```python
    def __init__(self, xs):
        self.n = len(xs)
        self.xs = xs
        self.tree = [None] * (4 * self.n)

    def add_line(self, m, b, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if lo > hi: return
        mid = (lo + hi) // 2
        x = self.xs[mid]
        new_line = (m, b)
        if self.tree[node] is None:
            self.tree[node] = new_line
            return
        cur = self.tree[node]
        if m * x + b < cur[0] * x + cur[1]:
            self.tree[node] = new_line
            new_line = cur
        m2, b2 = new_line
        if m2 * self.xs[lo] + b2 < self.tree[node][0]*self.xs[lo]+self.tree[node][1]:
            self.add_line(m2, b2, 2*node, lo, mid-1)
        else:
            self.add_line(m2, b2, 2*node+1, mid+1, hi)

    def query(self, x_idx, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if lo > hi: return float('inf')
        mid = (lo + hi) // 2
        best = float('inf')
        if self.tree[node]:
            m, b = self.tree[node]
            best = m * self.xs[x_idx] + b
        if x_idx <= mid:
            return min(best, self.query(x_idx, 2*node, lo, mid-1))
        else:
            return min(best, self.query(x_idx, 2*node+1, mid+1, hi))
```

Listing 50: Convex Hull Trick (Li Chao tree variant)

5 Greedy Algorithms

5.1 Interval Scheduling / Activity Selection

Algorithm 5.1 (Activity Selection)**.** Sort intervals by finish time. Greedily select each interval that starts after the last selected one finishes.

Why correct? Exchange argument: any solution can be transformed to the greedy one without reducing the count.

5.1.1 Python Implementation

```python
def activity_selection(intervals):
    """intervals = [(start, end)]. Returns max non-overlapping count."""
    intervals.sort(key=lambda x: x[1])
    count = 0
    last_end = float('-inf')
    selected = []
    for s, e in intervals:
        if s >= last_end:
            count += 1
            last_end = e
            selected.append((s, e))
    return count, selected

print(activity_selection([(1,3),(2,5),(3,9),(6,8)]))  # 3
```

Listing 51: Activity Selection – maximum non-overlapping intervals

Complexity. $\mathcal{O}(n \log n)$.

5.2 Huffman Coding

Algorithm 5.2 (Huffman)**.** Greedy prefix-free encoding. Repeatedly merge two nodes of lowest frequency. Produces optimal variable-length codes.

5.2.1 Python Implementation

```python
import heapq

def huffman(freq):
    """freq = {char: frequency}. Returns code lengths."""
    heap = [[f, [c, ""]] for c, f in freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        lo = heapq.heappop(heap)
        hi = heapq.heappop(heap)
        for pair in lo[1:]: pair[1] = '0' + pair[1]
        for pair in hi[1:]: pair[1] = '1' + pair[1]
        heapq.heappush(heap, [lo[0]+hi[0]] + lo[1:] + hi[1:])
    codes = {c: code for c, code in sorted(heap[0][1:], key=lambda x: x[0])}
    return codes

freq = {'a':5,'b':9,'c':12,'d':13,'e':16,'f':45}
print(huffman(freq))
```

Listing 52: Huffman Coding

Complexity. $\mathcal{O}(n \log n)$.

5.3 Greedy Exchange Argument

Template. To prove a greedy is optimal: assume an optimal solution differs from greedy; show swapping to match greedy doesn't worsen the solution. Repeat until solutions match.

Example. Job scheduling to minimize total weighted completion time: sort by w_i/t_i descending.

5.3.1 Python Implementation

```python
def weighted_job_schedule(jobs):
    """jobs = [(time, weight)]. Minimize sum of weight * completion_time."""
    # Sort by weight/time descending (or time/weight ascending)
    jobs.sort(key=lambda x: x[0] / x[1])  # ascending time/weight
    total_cost = 0
    cur_time = 0
    for t, w in jobs:
        cur_time += t
        total_cost += w * cur_time
    return total_cost
```

Listing 53: Weighted job scheduling – minimize weighted completion time

5.4 Fractional Knapsack

5.4.1 Python Implementation

```python
def fractional_knapsack(items, W):
    """items = [(weight, value)]. Returns max value (fractions allowed)."""
    items.sort(key=lambda x: x[1]/x[0], reverse=True)
    total = 0.0
    remaining = W
    for w, v in items:
        if remaining <= 0: break
        take = min(w, remaining)
        total += take * (v / w)
        remaining -= take
    return total

print(fractional_knapsack([(10,60),(20,100),(30,120)], 50))  # 240.0
```

Listing 54: Fractional Knapsack

Complexity. $\mathcal{O}(n \log n)$.

6 Data Structures

6.1 Segment Tree

Algorithm 6.1 (Segment Tree)**.** Binary tree where each node stores aggregate (sum/min/max) of a range. Supports point update and range query in $\mathcal{O}(\log n)$.

6.1.1 Python Implementation

```python
class SegTree:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (4 * n)

    def update(self, pos, val, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if lo == hi:
            self.tree[node] += val
            return
        mid = (lo + hi) // 2
        if pos <= mid: self.update(pos, val, 2*node, lo, mid)
        else:          self.update(pos, val, 2*node+1, mid+1, hi)
        self.tree[node] = self.tree[2*node] + self.tree[2*node+1]

    def query(self, l, r, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if r < lo or hi < l: return 0
        if l <= lo and hi <= r: return self.tree[node]
        mid = (lo + hi) // 2
        return (self.query(l, r, 2*node, lo, mid) +
                self.query(l, r, 2*node+1, mid+1, hi))

st = SegTree(8)
for i, v in enumerate([1,3,5,7,9,11,13,15]):
    st.update(i, v)
print(st.query(1, 5))  # 3+5+7+9+11 = 35
```

Listing 55: Segment Tree – range sum with point update

Complexity. $\mathcal{O}(\log n)$ update and query; $\mathcal{O}(n)$ build.

6.2 Lazy Propagation

6.2.1 Python Implementation

```python
class LazySegTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(arr, 1, 0, self.n - 1)

    def _build(self, arr, node, lo, hi):
        if lo == hi:
            self.tree[node] = arr[lo]; return
        mid = (lo + hi) // 2
        self._build(arr, 2*node, lo, mid)
        self._build(arr, 2*node+1, mid+1, hi)
        self.tree[node] = self.tree[2*node] + self.tree[2*node+1]

    def _push(self, node, lo, hi):
        if self.lazy[node]:
            mid = (lo + hi) // 2
            self.tree[2*node]   += self.lazy[node] * (mid - lo + 1)
```

```python
            self.tree[2*node+1] += self.lazy[node] * (hi - mid)
            self.lazy[2*node]   += self.lazy[node]
            self.lazy[2*node+1] += self.lazy[node]
            self.lazy[node] = 0

    def update(self, l, r, val, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if r < lo or hi < l: return
        if l <= lo and hi <= r:
            self.tree[node] += val * (hi - lo + 1)
            self.lazy[node] += val; return
        self._push(node, lo, hi)
        mid = (lo + hi) // 2
        self.update(l, r, val, 2*node, lo, mid)
        self.update(l, r, val, 2*node+1, mid+1, hi)
        self.tree[node] = self.tree[2*node] + self.tree[2*node+1]

    def query(self, l, r, node=1, lo=0, hi=None):
        if hi is None: hi = self.n - 1
        if r < lo or hi < l: return 0
        if l <= lo and hi <= r: return self.tree[node]
        self._push(node, lo, hi)
        mid = (lo + hi) // 2
        return (self.query(l, r, 2*node, lo, mid) +
                self.query(l, r, 2*node+1, mid+1, hi))
```

Listing 56: Segment Tree with Lazy Propagation – range add, range sum

6.3 Fenwick Tree (BIT)

Algorithm 6.2 (Binary Indexed Tree)**.** Efficiently computes prefix sums. Update and query in $\mathcal{O}(\log n)$ with very low constants.

6.3.1 Python Implementation

```python
class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)

    def update(self, i, delta=1):
        while i <= self.n:
            self.tree[i] += delta
            i += i & (-i)

    def query(self, i):
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & (-i)
        return s

    def range_query(self, l, r):
        return self.query(r) - self.query(l - 1)

    def kth(self, k):
        """Find k-th element (1-indexed)."""
        pos = 0
        log = self.n.bit_length()
        for i in range(log, -1, -1):
            npos = pos + (1 << i)
            if npos <= self.n and self.tree[npos] < k:
                pos = npos
                k -= self.tree[npos]
        return pos + 1
```

```python

# 2D BIT
class BIT2D:
    def __init__(self, n, m):
        self.n, self.m = n, m
        self.tree = [[0]*(m+1) for _ in range(n+1)]

    def update(self, x, y, delta=1):
        i = x
        while i <= self.n:
            j = y
            while j <= self.m:
                self.tree[i][j] += delta
                j += j & (-j)
            i += i & (-i)

    def query(self, x, y):
        s, i = 0, x
        while i > 0:
            j = y
            while j > 0:
                s += self.tree[i][j]
                j -= j & (-j)
            i -= i & (-i)
        return s
```

Listing 57: Fenwick Tree – prefix sum, range sum, 2D BIT

Complexity. $\mathcal{O}(\log n)$ per operation.

6.4 Sparse Table

Algorithm 6.3 (Sparse Table)**.** Precompute range-minimum (or idempotent aggregates) queries. $\mathcal{O}(n \log n)$ build, $\mathcal{O}(1)$ query.

6.4.1 Python Implementation

```python
import math

class SparseTable:
    def __init__(self, arr):
        n = len(arr)
        LOG = max(1, math.floor(math.log2(n)) + 1) if n > 1 else 1
        self.table = [arr[:]]
        for k in range(1, LOG):
            prev = self.table[k-1]
            cur = [min(prev[i], prev[i + (1<<(k-1))])
                   if i + (1<<k) - 1 < n else prev[i]
                   for i in range(n)]
            self.table.append(cur)
        self.log2 = [0] * (n + 1)
        for i in range(2, n + 1):
            self.log2[i] = self.log2[i//2] + 1

    def query(self, l, r):
        k = self.log2[r - l + 1]
        return min(self.table[k][l], self.table[k][r - (1<<k) + 1])

st = SparseTable([2,4,3,1,6,7,8,9,1,7])
print(st.query(2, 7))  # min of [3,1,6,7,8,9] = 1
```

Listing 58: Sparse Table for RMQ (range minimum query)

Complexity. $\mathcal{O}(n \log n)$ preprocess; $\mathcal{O}(1)$ query.

6.5 Trie

6.5.1 Python Implementation

```python
class TrieNode:
    __slots__ = ['children', 'count', 'end']
    def __init__(self):
        self.children = {}
        self.count = 0
        self.end = 0

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
            node.count += 1
        node.end += 1

    def search(self, word):
        node = self.root
        for c in word:
            if c not in node.children: return False
            node = node.children[c]
        return node.end > 0

    def starts_with(self, prefix):
        node = self.root
        for c in prefix:
            if c not in node.children: return 0
            node = node.children[c]
        return node.count

# Binary Trie for XOR problems
class BinaryTrie:
    def __init__(self, bits=30):
        self.root = [None, None]
        self.BITS = bits

    def insert(self, num):
        node = self.root
        for i in range(self.BITS, -1, -1):
            bit = (num >> i) & 1
            if not node[bit]: node[bit] = [None, None]
            node = node[bit]

    def max_xor(self, num):
        node = self.root
        res = 0
        for i in range(self.BITS, -1, -1):
            bit = (num >> i) & 1
            want = 1 - bit
            if node[want]:
                res |= (1 << i)
                node = node[want]
            elif node[bit]:
                node = node[bit]
            else:
                break
        return res
```

Listing 59: Trie – insert, search, prefix count

6.6 Monotonic Stack

Use. Next greater element; largest rectangle in histogram; stock span problem.

6.6.1 Python Implementation

```python
def next_greater(arr):
    n = len(arr)
    result = [-1] * n
    stack = []  # Stores indices
    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result

def largest_rectangle_histogram(heights):
    n = len(heights)
    stack = []
    max_area = 0
    for i in range(n + 1):
        h = heights[i] if i < n else 0
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    return max_area

print(largest_rectangle_histogram([2,1,5,6,2,3]))  # 10
```

Listing 60: Monotonic Stack – next greater element and histogram

6.7 Monotonic Queue (Deque)

Use. Sliding window minimum/maximum in $\mathcal{O}(n)$.

6.7.1 Python Implementation

```python
from collections import deque

def sliding_window_min(arr, k):
    dq = deque()  # Stores indices; front = min index
    result = []
    for i, x in enumerate(arr):
        while dq and arr[dq[-1]] >= x:
            dq.pop()
        dq.append(i)
        if dq[0] < i - k + 1:
            dq.popleft()
        if i >= k - 1:
            result.append(arr[dq[0]])
    return result

print(sliding_window_min([1,3,-1,-3,5,3,6,7], 3))
# [-1,-3,-3,-3,3,3]
```

Listing 61: Monotonic Deque – sliding window minimum

6.8 Priority Queue (Heap)

6.8.1 Python Implementation

```python
import heapq
```

```python

# Min-heap (default)
heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)
print(heapq.heappop(heap))  # 1

# Max-heap: negate values
max_heap = []
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -1)
print(-heapq.heappop(max_heap))  # 5

# K-th smallest in a stream
def kth_largest(stream, k):
    heap = []
    for x in stream:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]

# Lazy deletion heap
class LazyHeap:
    def __init__(self):
        self.heap = []
        self.removed = {}
    def push(self, val):
        heapq.heappush(self.heap, val)
    def remove(self, val):
        self.removed[val] = self.removed.get(val, 0) + 1
    def pop(self):
        while self.heap and self.removed.get(self.heap[0], 0) > 0:
            v = heapq.heappop(self.heap)
            self.removed[v] -= 1
        return heapq.heappop(self.heap)
```

Listing 62: Priority Queue patterns in Python

6.9 Hash Map / Hashing

6.9.1 Python Implementation

```python
from collections import defaultdict, Counter

# Frequency counting
def most_frequent(arr):
    return Counter(arr).most_common(1)[0]

# Group by hash
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)
    return list(groups.values())

# Polynomial hashing (anti-hack: use double hash)
class PolyHash:
    def __init__(self, s, base=131, mod=10**9+7):
        n = len(s)
        self.mod = mod
        self.h = [0] * (n + 1)
        self.pw = [1] * (n + 1)
        for i, c in enumerate(s):
```

```
            self.h[i+1] = (self.h[i] * base + ord(c)) % mod
            self.pw[i+1] = self.pw[i] * base % mod

    def get(self, l, r):  # 0-indexed, inclusive
        return (self.h[r+1] - self.h[l] * self.pw[r-l+1]) % self.mod
```

Listing 63: Hashing patterns for competitive programming

7 String Algorithms

7.1 KMP Algorithm

Algorithm 7.1 (Knuth-Morris-Pratt)**.** Find all occurrences of pattern P in text T in $\mathcal{O}(|T| + |P|)$ using failure function.

7.1.1 Python Implementation

```python
def kmp_fail(pattern):
    n = len(pattern)
    fail = [0] * n
    j = 0
    for i in range(1, n):
        while j > 0 and pattern[i] != pattern[j]:
            j = fail[j-1]
        if pattern[i] == pattern[j]:
            j += 1
        fail[i] = j
    return fail

def kmp_search(text, pattern):
    """Returns list of start indices of pattern in text."""
    fail = kmp_fail(pattern)
    result = []
    j = 0
    for i, c in enumerate(text):
        while j > 0 and c != pattern[j]:
            j = fail[j-1]
        if c == pattern[j]:
            j += 1
        if j == len(pattern):
            result.append(i - len(pattern) + 1)
            j = fail[j-1]
    return result

print(kmp_search("ababcababc", "ababc"))  # [0, 5]
```

Listing 64: KMP – failure function and pattern matching

Complexity. $\mathcal{O}(|T| + |P|)$.

7.2 Z-Algorithm

7.2.1 Python Implementation

```python
def z_function(s):
    n = len(s)
    z = [0] * n
    z[0] = n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z

def z_search(text, pattern):
    """Find pattern in text using Z-algorithm."""
    s = pattern + '$' + text
    z = z_function(s)
```

```python
    m = len(pattern)
    return [i - m - 1 for i in range(m+1, len(s)) if z[i] == m]

print(z_search("ababcababc", "ababc"))  # [0, 5]
```

Listing 65: = longest match of s[i:] with prefix]Z-Function – z[i] = longest match of s[i:] with prefix

Complexity. $\mathcal{O}(n)$.

7.3 Suffix Array

7.3.1 Python Implementation

```python
def build_suffix_array(s):
    """O(n log^2 n) rank-pair sorting construction."""
    n = len(s)
    sa = list(range(n))
    rank = [ord(c) for c in s]
    tmp = [0] * n
    k = 1
    while k < n:
        key = lambda i: (rank[i], rank[i+k] if i+k < n else -1)
        sa.sort(key=key)
        tmp[sa[0]] = 0
        for i in range(1, n):
            tmp[sa[i]] = tmp[sa[i-1]]
            if key(sa[i]) != key(sa[i-1]):
                tmp[sa[i]] += 1
        rank = tmp[:]
        if rank[sa[-1]] == n - 1: break
        k <<= 1
    return sa

def build_lcp(s, sa):
    """Kasai's algorithm: LCP array in O(n)."""
    n = len(s)
    rank = [0] * n
    for i, v in enumerate(sa):
        rank[v] = i
    lcp = [0] * n
    h = 0
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i]-1]
            while i+h < n and j+h < n and s[i+h] == s[j+h]:
                h += 1
            lcp[rank[i]] = h
            if h > 0: h -= 1
    return lcp

s = "banana"
sa = build_suffix_array(s)
lcp = build_lcp(s, sa)
print(sa)   # [5,3,1,0,4,2]
print(lcp)  # [0,1,3,0,0,2]
```

Listing 66: Suffix Array – $O(n \log^2 n)$ construction

Complexity. Build: $\mathcal{O}(n \log^2 n)$; LCP: $\mathcal{O}(n)$.

7.4 Rolling Hash / Rabin-Karp

7.4.1 Python Implementation

```python
def rabin_karp(text, pattern, base=131, mod=10**9+7):
```

```python
    n, m = len(text), len(pattern)
    if m > n: return []
    pm = bm = 0
    power = 1
    for i in range(m):
        pm = (pm * base + ord(pattern[i])) % mod
        bm = (bm * base + ord(text[i])) % mod
        if i > 0: power = power * base % mod
    result = []
    if pm == bm and text[:m] == pattern:
        result.append(0)
    for i in range(1, n - m + 1):
        bm = (bm - ord(text[i-1]) * power % mod + mod) % mod
        bm = (bm * base + ord(text[i+m-1])) % mod
        if bm == pm and text[i:i+m] == pattern:
            result.append(i)
    return result
```

Listing 67: Rabin-Karp rolling hash for pattern matching

Complexity. $\mathcal{O}(nm)$ worst case; $\mathcal{O}(n+m)$ expected.

7.5 Manacher's Algorithm

Algorithm 7.2 (Manacher)**.** Finds longest palindromic substring (and all palindrome radii) in $\mathcal{O}(n)$.

7.5.1 Python Implementation

```python
def manacher(s):
    """Returns p[] where p[i] = radius of palindrome centered at i in transformed string."""
    t = '#' + '#'.join(s) + '#'
    n = len(t)
    p = [0] * n
    c = r = 0
    for i in range(n):
        mirror = 2*c - i
        if i < r:
            p[i] = min(r - i, p[mirror])
        while i + p[i] + 1 < n and i - p[i] - 1 >= 0 and t[i+p[i]+1] == t[i-p[i]-1]:
            p[i] += 1
        if i + p[i] > r:
            c, r = i, i + p[i]
    return p

def longest_palindrome(s):
    p = manacher(s)
    max_len, center = max((v, i) for i, v in enumerate(p))
    start = (center - max_len) // 2
    return s[start:start + max_len]

print(longest_palindrome("babad"))  # "bab" or "aba"
```

Listing 68: Manacher's Algorithm

Complexity. $\mathcal{O}(n)$.

7.6 Trie-based Matching

Already shown in Section 6 (Trie). For Aho-Corasick (multi-pattern matching):

7.6.1 Python Implementation

```python
from collections import deque

def aho_corasick(patterns, text):
    # Build trie
    goto = [{}]
    fail = [0]
    output = [[]]

    for pat in patterns:
        cur = 0
        for c in pat:
            if c not in goto[cur]:
                goto[cur][c] = len(goto)
                goto.append({})
                fail.append(0)
                output.append([])
            cur = goto[cur][c]
        output[cur].append(pat)

    # Build failure links via BFS
    q = deque()
    for c, s in goto[0].items():
        fail[s] = 0
        q.append(s)
    while q:
        r = q.popleft()
        for c, s in goto[r].items():
            q.append(s)
            state = fail[r]
            while state and c not in goto[state]:
                state = fail[state]
            fail[s] = goto[state].get(c, 0)
            if fail[s] == s: fail[s] = 0
            output[s] += output[fail[s]]

    # Search
    cur = 0
    results = []
    for i, c in enumerate(text):
        while cur and c not in goto[cur]:
            cur = fail[cur]
        cur = goto[cur].get(c, 0)
        for pat in output[cur]:
            results.append((i - len(pat) + 1, pat))
    return results
```

Listing 69: Aho-Corasick multi-pattern matching

8 Geometry

8.1 Cross Product & Dot Product

Definition 8.1. For 2D vectors $\vec{a} = (a_x, a_y)$ and $\vec{b} = (b_x, b_y)$:

$$\vec{a} \times \vec{b} = a_x b_y - a_y b_x \quad \text{(scalar, sign = orientation)}$$

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y \quad \text{(scalar, sign = angle relationship)}$$

8.1.1 Python Implementation

```python
def cross(O, A, B):
    """Signed area of triangle OAB * 2. >0: CCW, <0: CW, =0: collinear."""
    return (A[0]-O[0])*(B[1]-O[1]) - (A[1]-O[1])*(B[0]-O[0])

def dot(O, A, B):
    return (A[0]-O[0])*(B[0]-O[0]) + (A[1]-O[1])*(B[1]-O[1])

def orientation(P, Q, R):
    val = cross(P, Q, R)
    if val == 0: return 0   # Collinear
    return 1 if val > 0 else -1  # 1=CCW, -1=CW

def dist2(A, B):
    """Squared distance (avoids float)."""
    return (A[0]-B[0])**2 + (A[1]-B[1])**2
```

Listing 70: Cross product, dot product, orientation test

8.2 Convex Hull (Andrew's Monotone Chain)

8.2.1 Python Implementation

```python
def convex_hull(points):
    points = sorted(set(points))
    if len(points) <= 1: return points
    lower = []
    for p in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)
    upper = []
    for p in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]

def cross(O, A, B):
    return (A[0]-O[0])*(B[1]-O[1]) - (A[1]-O[1])*(B[0]-O[0])

pts = [(0,0),(1,1),(2,2),(0,2),(2,0)]
hull = convex_hull(pts)
print(hull)  # [(0,0),(2,0),(2,2),(0,2)]
```

Listing 71: Convex Hull – Andrew's Monotone Chain O(n log n)

Complexity. $\mathcal{O}(n \log n)$.

8.3 Shoelace Formula & Polygon Area

$$\text{Area} = \frac{1}{2}\left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

8.3.1 Python Implementation

```python
def polygon_area(pts):
    """Signed area * 2 (integer). Positive = CCW."""
    n = len(pts)
    area2 = 0
    for i in range(n):
        x1, y1 = pts[i]
        x2, y2 = pts[(i+1) % n]
        area2 += x1 * y2 - x2 * y1
    return abs(area2) / 2

print(polygon_area([(0,0),(4,0),(4,3),(0,3)]))  # 12.0
```

Listing 72: Shoelace formula for polygon area

8.4 Pick's Theorem

Theorem 8.2 (Pick's Theorem)**.** For a lattice polygon with integer vertices:

$$A = I + \frac{B}{2} - 1,$$

where A = area, I = interior lattice points, B = boundary lattice points.

8.4.1 Python Implementation

```python
from math import gcd

def boundary_points(pts):
    """Count lattice points on boundary of polygon."""
    n = len(pts)
    b = 0
    for i in range(n):
        x1, y1 = pts[i]
        x2, y2 = pts[(i+1) % n]
        b += gcd(abs(x2-x1), abs(y2-y1))
    return b

def interior_points(pts):
    area2 = int(sum((pts[i][0]*pts[(i+1)%len(pts)][1] -
                     pts[(i+1)%len(pts)][0]*pts[i][1])
                    for i in range(len(pts))))
    area2 = abs(area2)
    B = boundary_points(pts)
    return (area2 - B + 2) // 2  # Pick: I = A - B/2 + 1
```

Listing 73: Pick's Theorem – interior lattice points

8.5 Line Intersection

8.5.1 Python Implementation

```python
def on_segment(P, Q, R):
    """Is R on segment PQ (assuming collinear)?"""
    return (min(P[0],Q[0]) <= R[0] <= max(P[0],Q[0]) and
            min(P[1],Q[1]) <= R[1] <= max(P[1],Q[1]))

```

```python
def segments_intersect(P1, Q1, P2, Q2):
    d1 = cross(P2, Q2, P1)
    d2 = cross(P2, Q2, Q1)
    d3 = cross(P1, Q1, P2)
    d4 = cross(P1, Q1, Q2)
    if ((d1>0 and d2<0 or d1<0 and d2>0) and
        (d3>0 and d4<0 or d3<0 and d4>0)):
        return True
    if d1==0 and on_segment(P2, Q2, P1): return True
    if d2==0 and on_segment(P2, Q2, Q1): return True
    if d3==0 and on_segment(P1, Q1, P2): return True
    if d4==0 and on_segment(P1, Q1, Q2): return True
    return False

def cross(O, A, B):
    return (A[0]-O[0])*(B[1]-O[1]) - (A[1]-O[1])*(B[0]-O[0])

def line_intersection(P1, P2, P3, P4):
    """Intersection point of lines P1P2 and P3P4 (if exists)."""
    d1x, d1y = P2[0]-P1[0], P2[1]-P1[1]
    d2x, d2y = P4[0]-P3[0], P4[1]-P3[1]
    denom = d1x*d2y - d1y*d2x
    if denom == 0: return None  # Parallel
    t = ((P3[0]-P1[0])*d2y - (P3[1]-P1[1])*d2x) / denom
    return (P1[0] + t*d1x, P1[1] + t*d1y)
```

Listing 74: Line segment intersection

8.6 Point in Polygon

8.6.1 Python Implementation

```python
def point_in_polygon(point, polygon):
    """Ray casting algorithm. Returns True if point is inside polygon."""
    x, y = point
    n = len(polygon)
    inside = False
    px, py = polygon[-1]
    for qx, qy in polygon:
        if ((py > y) != (qy > y) and
                x < (qx - px) * (y - py) / (qy - py) + px):
            inside = not inside
        px, py = qx, qy
    return inside

# Winding number (more robust)
def winding_number(point, polygon):
    x, y = point
    wn = 0
    n = len(polygon)
    for i in range(n):
        x1, y1 = polygon[i]
        x2, y2 = polygon[(i+1) % n]
        if y1 <= y:
            if y2 > y:
                if cross((x1,y1),(x2,y2),(x,y)) > 0:
                    wn += 1
        else:
            if y2 <= y:
                if cross((x1,y1),(x2,y2),(x,y)) < 0:
                    wn -= 1
    return wn  # 0 = outside

def cross(O, A, B):
    return (A[0]-O[0])*(B[1]-O[1]) - (A[1]-O[1])*(B[0]-O[0])
```

Listing 75: Point-in-polygon ray casting

8.7 Distance Formulas

Mathematical formulations.

- **Euclidean:** $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- **Manhattan:** $d = |x_2 - x_1| + |y_2 - y_1|$ (grid, taxicab)
- **Chebyshev:** $d = \max(|x_2 - x_1|, |y_2 - y_1|)$ (king moves)
- **Chebyshev $\leftrightarrow$ Manhattan trick:** $(x, y) \to (x + y, x - y)$ converts Chebyshev to Manhattan and vice versa.

CP Utility: Grid problems; closest pair; geometry distance queries; metric space arguments.

8.7.1 Python Implementation

```python
import math

def euclidean(P, Q):
    return math.hypot(P[0]-Q[0], P[1]-Q[1])

def manhattan(P, Q):
    return abs(P[0]-Q[0]) + abs(P[1]-Q[1])

def chebyshev(P, Q):
    return max(abs(P[0]-Q[0]), abs(P[1]-Q[1]))

def chebyshev_to_manhattan(points):
    """Transform so Chebyshev dist becomes Manhattan."""
    return [(x+y, x-y) for x, y in points]

def closest_pair(pts):
    """Closest pair of points in O(n log n)."""
    pts.sort()
    def rec(P):
        if len(P) <= 3:
            return min((euclidean(P[i],P[j]), P[i], P[j])
                       for i in range(len(P))
                       for j in range(i+1, len(P)))
        mid = len(P) // 2
        mx = P[mid][0]
        d, *pair = min(rec(P[:mid]), rec(P[mid:]))
        strip = [p for p in P if abs(p[0]-mx) < d]
        strip.sort(key=lambda p: p[1])
        for i in range(len(strip)):
            j = i + 1
            while j < len(strip) and strip[j][1]-strip[i][1] < d:
                nd = euclidean(strip[i], strip[j])
                if nd < d:
                    d = nd; pair = [strip[i], strip[j]]
                j += 1
        return (d, *pair)
    return rec(pts)

print(closest_pair([(0,0),(1,1),(3,4),(2,2)]))  # closest pair
```

Listing 76: Distance formulas and closest pair

Complexity. Individual distances: $\mathcal{O}(1)$; Closest pair: $\mathcal{O}(n \log n)$.

9 Probability & Mathematics

9.1 Expected Value & Linearity of Expectation

Theorem 9.1 (Linearity of Expectation). For any random variables $X_1, \ldots, X_n$ (not necessarily independent):

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

Intuition. Decompose complex random variables into simple indicator variables.

Classic application. Expected number of comparisons in randomized algorithms; expected cycle length in random permutations.

9.1.1 Python Implementation

```python
def expected_hats_returned(n):
    """Expected fixed points in a random permutation of n elements."""
    # By linearity: E[X] = sum E[X_i] = n * (1/n) = 1
    return 1.0  # Always 1, regardless of n!

def expected_steps(P, start, target):
    """Expected steps from start to absorbing target state.
    P[i] = [(next_state, probability), ...]. Gaussian elim."""
    n = len(P)
    # System: E[i] - sum_j p_ij*E[j] = 1 (i!=target); E[target]=0
    A = [[0.0]*(n+1) for _ in range(n)]
    for i in range(n):
        if i == target:
            A[i][i] = 1.0
        else:
            A[i][i] = 1.0; A[i][n] = 1.0
            for j, p in P[i]:
                A[i][j] -= p
    for col in range(n):  # Forward elimination
        for row in range(col + 1, n):
            if abs(A[col][col]) < 1e-12: continue
            f = A[row][col] / A[col][col]
            for k in range(n + 1):
                A[row][k] -= f * A[col][k]
    E = [0.0] * n          # Back substitution
    for i in range(n - 1, -1, -1):
        E[i] = A[i][n]
        for j in range(i + 1, n):
            E[i] -= A[i][j] * E[j]
        if abs(A[i][i]) > 1e-12:
            E[i] /= A[i][i]
    return E[start]

# Gambler's ruin: states 0..4, absorb at 0 and 4
# P[i] = [(i-1, 0.5), (i+1, 0.5)] for 1<=i<=3
# expected_steps(P, 2, 4)  # expected steps from 2 to 4
```

Listing 77: Expected value via linearity – hat-check problem

9.2 Bayes' Theorem

Theorem 9.2 (Bayes' Theorem). For hypotheses $H_1, \ldots, H_n$ forming a partition:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}, \qquad P(H_i \mid B) = \frac{P(B \mid H_i)\, P(H_i)}{\sum_j P(B \mid H_j)\, P(H_j)}.$$

Intuition. Bayes inverts conditional probabilities: given $P(\text{evidence} \mid \text{hypothesis})$, compute $P(\text{hypothesis} \mid \text{evidence})$.

CP Utility: Probability problems with conditional information; interactive problems with Bayesian updates; classification scenarios.

Example Problem. Two bags: Bag 1 has 3 red, 2 blue balls; Bag 2 has 1 red, 4 blue. A bag is chosen uniformly, then a ball is drawn. Given the ball is red, what is the probability it came from Bag 1?

9.2.1 Python Implementation

```python
def bayes(prior, likelihood):
    """prior[i] = P(H_i), likelihood[i] = P(evidence | H_i).
    Returns posterior[i] = P(H_i | evidence)."""
    joint = [prior[i] * likelihood[i] for i in range(len(prior))]
    total = sum(joint)
    return [j / total for j in joint]

# Bag 1: 3R/5, Bag 2: 1R/5. Prior = [0.5, 0.5]
prior = [0.5, 0.5]
likelihood = [3/5, 1/5]
print(bayes(prior, likelihood))  # [0.75, 0.25]
```

Listing 78: Bayes' Theorem – posterior computation

Complexity. $\mathcal{O}(n)$ for n hypotheses.

9.3 Randomized Algorithms

Definition 9.3. A **Monte Carlo** algorithm gives the correct answer with high probability (may be wrong). A **Las Vegas** algorithm always gives the correct answer but has random running time.

9.3.1 Python Implementation

```python
import random

def miller_rabin(n, k=20):
    """Probabilistic primality test. Error prob < 4^(-k)."""
    if n < 2: return False
    if n == 2 or n == 3: return True
    if n % 2 == 0: return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1; d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = x * x % n
            if x == n - 1: break
        else:
            return False
    return True

def deterministic_miller_rabin(n):
    """Deterministic for n < 3.2*10^18."""
    witnesses = [2,3,5,7,11,13,17,19,23,29,31,37]
    if n < 2: return False
    if n in witnesses: return True
    if any(n % w == 0 for w in witnesses): return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1; d //= 2
    for a in witnesses:
        if a >= n: continue
        x = pow(a, d, n)
```

```python
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = x * x % n
            if x == n - 1: break
        else:
            return False
    return True

print(deterministic_miller_rabin(998244353))  # True (prime)
```

Listing 79: Randomized algorithms – Miller-Rabin primality and random hash

Complexity. $\mathcal{O}(k \log^2 n)$ per test.

9.4 Randomized Techniques in CP

- **Randomized hashing:** Use random base/mod to avoid adversarial anti-hash tests.
- **Random shuffling:** Shuffle input before applying deterministic algorithm (Quicksort).
- **Random sampling:** Estimate quantities via Monte Carlo.

9.4.1 Python Implementation

```python
import random

class AntiHackHash:
    def __init__(self, s):
        MOD1 = (1 << 61) - 1  # Mersenne prime
        MOD2 = 10**9 + 9
        BASE1 = random.randint(131, 1000000)
        BASE2 = random.randint(137, 1000000)
        n = len(s)
        self.h1 = [0]*(n+1); self.h2 = [0]*(n+1)
        self.p1 = [1]*(n+1); self.p2 = [1]*(n+1)
        for i, c in enumerate(s):
            self.h1[i+1] = (self.h1[i]*BASE1 + ord(c)) % MOD1
            self.h2[i+1] = (self.h2[i]*BASE2 + ord(c)) % MOD2
            self.p1[i+1] = self.p1[i]*BASE1 % MOD1
            self.p2[i+1] = self.p2[i]*BASE2 % MOD2
        self.M1, self.M2 = MOD1, MOD2

    def get(self, l, r):  # 0-indexed inclusive
        v1 = (self.h1[r+1] - self.h1[l]*self.p1[r-l+1]) % self.M1
        v2 = (self.h2[r+1] - self.h2[l]*self.p2[r-l+1]) % self.M2
        return (v1, v2)
```

Listing 80: Anti-hack double hashing for strings

10 CP Patterns

10.1 Sliding Window

Pattern. Maintain a window $[l, r]$ that expands right and contracts left. $\mathcal{O}(n)$ for fixed-size or variable-size windows.

10.1.1 Python Implementation

```python
def max_subarray_fixed(arr, k):
    n = len(arr)
    win_sum = sum(arr[:k])
    best = win_sum
    for i in range(k, n):
        win_sum += arr[i] - arr[i-k]
        best = max(best, win_sum)
    return best

def min_subarray_sum_geq_s(arr, s):
    """Minimum length subarray with sum >= s."""
    n = len(arr)
    best = float('inf')
    cur = l = 0
    for r in range(n):
        cur += arr[r]
        while cur >= s:
            best = min(best, r - l + 1)
            cur -= arr[l]
            l += 1
    return best if best != float('inf') else 0

def longest_no_repeat(s):
    """Longest substring without repeating characters."""
    last = {}
    l = best = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        best = max(best, r - l + 1)
        last[c] = r
    return best
```

Listing 81: Sliding Window – maximum subarray sum of length k

10.2 Two Pointers

10.2.1 Python Implementation

```python
def two_sum_sorted(arr, target):
    """Find pair summing to target in sorted array."""
    l, r = 0, len(arr)-1
    while l < r:
        s = arr[l] + arr[r]
        if s == target: return (l, r)
        elif s < target: l += 1
        else: r -= 1
    return None

def three_sum(arr):
    """All unique triplets summing to 0."""
    arr.sort()
    n = len(arr)
    result = []
    for i in range(n-2):
```

```python
        if i > 0 and arr[i] == arr[i-1]: continue
        l, r = i+1, n-1
        while l < r:
            s = arr[i] + arr[l] + arr[r]
            if s == 0:
                result.append([arr[i], arr[l], arr[r]])
                while l < r and arr[l]==arr[l+1]: l+=1
                while l < r and arr[r]==arr[r-1]: r-=1
                l+=1; r-=1
            elif s < 0: l+=1
            else: r-=1
    return result
```

Listing 82: Two Pointers – pair sum, three sum

10.3 Binary Search on Answer

Pattern. If the answer satisfies a monotone predicate (false, false, ..., true, true), binary search on the answer space.

10.3.1 Python Implementation

```python
def binary_search_answer(lo, hi, check):
    """Find smallest x in [lo, hi] where check(x) is True."""
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

# Example: koko eats bananas
def min_eating_speed(piles, H):
    import math
    def can_finish(k):
        return sum(math.ceil(p/k) for p in piles) <= H
    return binary_search_answer(1, max(piles), can_finish)

# Example: allocate minimum pages
def allocate_books(pages, m):
    def feasible(max_pages):
        students = 1; cur = 0
        for p in pages:
            if p > max_pages: return False
            if cur + p > max_pages:
                students += 1; cur = 0
            cur += p
        return students <= m
    return binary_search_answer(max(pages), sum(pages), feasible)
```

Listing 83: Binary search on answer – minimize maximum

10.4 Prefix Sums

10.4.1 Python Implementation

```python
def prefix_sum_1d(arr):
    n = len(arr)
    pre = [0] * (n + 1)
    for i, v in enumerate(arr):
        pre[i+1] = pre[i] + v
    return pre

```

```python
def range_sum(pre, l, r):  # 0-indexed inclusive
    return pre[r+1] - pre[l]

# 2D prefix sum
def prefix_sum_2d(grid):
    n, m = len(grid), len(grid[0])
    pre = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n):
        for j in range(m):
            pre[i+1][j+1] = (grid[i][j] + pre[i][j+1] +
                             pre[i+1][j] - pre[i][j])
    return pre

def rect_sum(pre, r1, c1, r2, c2):
    return (pre[r2+1][c2+1] - pre[r1][c2+1] -
            pre[r2+1][c1] + pre[r1][c1])
```

Listing 84: Prefix Sums – 1D, 2D, and difference arrays

10.5 Difference Arrays

10.5.1 Python Implementation

```python
def range_add(diff, l, r, val):
    diff[l] += val
    if r + 1 < len(diff): diff[r+1] -= val

def apply_diff(diff):
    arr = diff[:]
    for i in range(1, len(arr)):
        arr[i] += arr[i-1]
    return arr

# Example: add val to [l,r] for multiple queries, then retrieve
n = 10
diff = [0] * (n + 1)
range_add(diff, 1, 4, 3)
range_add(diff, 2, 6, -1)
result = apply_diff(diff)
print(result[:n])  # [0,3,2,2,2,-1,-1,0,0,0]
```

Listing 85: Difference Arrays – range update in O(1)

10.6 Coordinate Compression

10.6.1 Python Implementation

```python
def compress(arr):
    """Map values to indices 0..k-1."""
    sorted_unique = sorted(set(arr))
    rank = {v: i for i, v in enumerate(sorted_unique)}
    return [rank[v] for v in arr], sorted_unique

def compress_2d(points):
    xs = sorted(set(p[0] for p in points))
    ys = sorted(set(p[1] for p in points))
    rx = {v: i for i, v in enumerate(xs)}
    ry = {v: i for i, v in enumerate(ys)}
    return [(rx[x], ry[y]) for x, y in points]

arr = [100, 200, 150, 300, 200]
comp, vals = compress(arr)
print(comp)  # [0, 2, 1, 3, 2]
print(vals)  # [100, 150, 200, 300]
```

Listing 86: Coordinate Compression

10.7 Mo's Algorithm

Algorithm 10.1 (Mo's Algorithm)**.** Answer offline range queries in $\mathcal{O}((n+q)\sqrt{n})$ by sorting queries by block.

10.7.1 Python Implementation

```python
def mo_algorithm(arr, queries):
    """queries = [(l, r, idx)]. Returns answers[]."""
    n = len(arr)
    block = max(1, int(n**0.5))
    queries = sorted(queries,
                     key=lambda q: (q[0]//block,
                                    q[1] if (q[0]//block)%2==0 else -q[1]))
    cnt = {}
    cur_ans = 0
    ans = [0] * len(queries)

    def add(x):
        nonlocal cur_ans
        cnt[x] = cnt.get(x, 0) + 1
        # Update cur_ans (problem-specific)

    def remove(x):
        nonlocal cur_ans
        cnt[x] -= 1
        if cnt[x] == 0: del cnt[x]
        # Update cur_ans

    l = r = 0
    add(arr[0])
    for ql, qr, qi in queries:
        while r < qr: r += 1; add(arr[r])
        while l > ql: l -= 1; add(arr[l])
        while r > qr: remove(arr[r]); r -= 1
        while l < ql: remove(arr[l]); l += 1
        ans[qi] = cur_ans
    return ans
```

Listing 87: Mo's Algorithm for offline range queries

Complexity. $\mathcal{O}((n+q)\sqrt{n})$.

10.8 Offline Queries

Pattern. Sort queries and process them in a convenient order (by right endpoint, by value, etc.) using an online data structure (BIT, segment tree).

10.8.1 Python Implementation

```python
def offline_distinct(arr, queries):
    """Count distinct elements in [l,r] for each query."""
    n = len(arr)
    # Process queries offline sorted by right endpoint
    qs = sorted(enumerate(queries), key=lambda x: x[1][1])
    bit = [0] * (n + 1)

    def update(i, v):
        while i <= n:
            bit[i] += v; i += i & -i
```

```python

    def query(i):
        s = 0
        while i > 0:
            s += bit[i]; i -= i & -i
        return s

    last = {}
    ans = [0] * len(queries)
    qi = 0
    for r in range(n):
        x = arr[r]
        if x in last:
            update(last[x]+1, -1)
        update(r+1, 1)
        last[x] = r
        while qi < len(qs) and qs[qi][1][1] == r:
            orig_i, (l, _) = qs[qi]
            ans[orig_i] = query(r+1) - query(l)
            qi += 1
    return ans
```

Listing 88: Offline range distinct count using BIT

10.9 Bitmasking

10.9.1 Python Implementation

```python
# Common bitmask operations
def bits(mask, n):
    """Iterate set bits."""
    return [i for i in range(n) if mask >> i & 1]

def lowest_bit(x): return x & (-x)

def pop_lowest(x): return x & (x - 1)

def submasks(mask):
    s = mask
    while s:
        yield s
        s = (s - 1) & mask

# Check if exactly k bits set
def has_k_bits(x, k): return bin(x).count('1') == k

# Turn on/off bits
def set_bit(x, i):    return x | (1 << i)
def clear_bit(x, i):  return x & ~(1 << i)
def toggle_bit(x, i): return x ^ (1 << i)
def check_bit(x, i):  return bool(x >> i & 1)

# Iterate all subsets of size k
from itertools import combinations
def subsets_of_size(n, k):
    for bits in combinations(range(n), k):
        yield sum(1 << b for b in bits)
```

Listing 89: Bitmasking tricks for competitive programming

10.10 Subset Iteration Trick (SOS DP)

10.10.1 Python Implementation

```python
def sos_dp(f, n):
    """f[mask] = sum of f[submask] for all submasks of mask.
    O(n * 2^n)."""
    for i in range(n):
        for mask in range(1 << n):
            if mask >> i & 1:
                f[mask] += f[mask ^ (1 << i)]
    return f

# Example: f[mask] = number of submasks with property P
n = 4
f = [1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]  # 2^4
sos_dp(f, n)
# f[mask] now = count of submasks that have property P
```

Listing 90: SOS (Sum over Subsets) DP

Complexity. $\mathcal{O}(n \cdot 2^n)$.

10.11 Meet-in-the-Middle

Pattern. Split the input in half ($n/2$ each), enumerate all possibilities for each half (e.g., $2^{n/2}$ subsets), then combine efficiently using sorting + binary search or two pointers.

CP Utility: Reduces exponential brute force from $\mathcal{O}(2^n)$ to $\mathcal{O}\big(2^{n/2} \log n\big)$. Critical when $n \leq 40$.

Where to use. Subset sum with $n \leq 40$; 4-SUM; knapsack with large n but small halves.

10.11.1 Python Implementation

```python
from bisect import bisect_left, bisect_right

def mitm_count(arr, target):
    """Count subsets of arr that sum to target. n <= 40."""
    n = len(arr)
    half = n // 2
    left, right = arr[:half], arr[half:]

    def all_sums(a):
        sums = []
        for mask in range(1 << len(a)):
            sums.append(sum(a[i] for i in range(len(a)) if mask >> i & 1))
        return sorted(sums)

    left_s = all_sums(left)
    right_s = all_sums(right)
    count = 0
    for s in left_s:
        need = target - s
        count += bisect_right(right_s, need) - bisect_left(right_s, need)
    return count

print(mitm_count([1,2,3,4,5,6], 10))  # subsets summing to 10
```

Listing 91: Meet-in-the-Middle – count subsets summing to target

Complexity. $\mathcal{O}\big(2^{n/2} \log n\big)$.

10.12 Fast I/O Template

Why it matters. Python's default `input()` is slow. For problems with $n > 10^5$, always use fast I/O to avoid TLE.

10.12.1 Python Implementation

```python
import sys
input = sys.stdin.readline

def main():
    # Read entire input at once (fastest)
    data = sys.stdin.buffer.read().split()
    idx = 0
    def rd(): nonlocal idx; idx += 1; return int(data[idx-1])

    n = rd()
    arr = [rd() for _ in range(n)]

    # Process...
    ans = sum(arr)

    # Fast output
    sys.stdout.write(str(ans) + '\n')

main()

# Alternative: for multiple test cases
def solve():
    import sys
    input = sys.stdin.readline
    out = []
    T = int(input())
    for _ in range(T):
        n = int(input())
        arr = list(map(int, input().split()))
        out.append(str(sum(arr)))
    print('\n'.join(out))
```

Listing 92: Fast I/O template for competitive programming

Note / Pitfall: In Python, `sys.stdin.buffer.read()` into a byte buffer and splitting once is $5-10\times$ faster than repeated `input()` calls. Always use this for $n > 10^5$.

Quick Reference Complexity Table

Algorithm	Time	Space
Sieve of Eratosthenes	$\mathcal{O}(n \log \log n)$	$\mathcal{O}(n)$
GCD (Euclidean)	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Binary Exponentiation	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
BFS / DFS	$\mathcal{O}(V + E)$	$\mathcal{O}(V)$
Dijkstra	$\mathcal{O}((V + E) \log V)$	$\mathcal{O}(V + E)$
Bellman-Ford	$\mathcal{O}(VE)$	$\mathcal{O}(V)$
Floyd-Warshall	$\mathcal{O}(V^3)$	$\mathcal{O}(V^2)$
Kruskal	$\mathcal{O}(E \log E)$	$\mathcal{O}(V)$
Tarjan/Kosaraju SCC	$\mathcal{O}(V + E)$	$\mathcal{O}(V)$
Dinic Max Flow	$\mathcal{O}(V^2 E)$	$\mathcal{O}(V + E)$
LCA Binary Lifting	$\mathcal{O}(N \log N)$ / $\mathcal{O}(\log N)$	$\mathcal{O}(N \log N)$
Segment Tree	$\mathcal{O}(n)$ build, $\mathcal{O}(\log n)$ q/u	$\mathcal{O}(n)$
Fenwick Tree	$\mathcal{O}(\log n)$ q/u	$\mathcal{O}(n)$
Sparse Table	$\mathcal{O}(n \log n)$ / $\mathcal{O}(1)$	$\mathcal{O}(n \log n)$
KMP	$\mathcal{O}(n + m)$	$\mathcal{O}(m)$
Z-Algorithm	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Suffix Array	$\mathcal{O}(n \log^2 n)$	$\mathcal{O}(n)$
Manacher	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Convex Hull	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Closest Pair	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Interval DP	$\mathcal{O}(n^3)$	$\mathcal{O}(n^2)$
0/1 Knapsack	$\mathcal{O}(nW)$	$\mathcal{O}(W)$
LIS	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Bitmask DP (TSP)	$\mathcal{O}(2^n n^2)$	$\mathcal{O}(2^n n)$
Mo's Algorithm	$\mathcal{O}((n + q)\sqrt{n})$	$\mathcal{O}(n + q)$
SOS DP	$\mathcal{O}(n \cdot 2^n)$	$\mathcal{O}(2^n)$
Miller-Rabin	$\mathcal{O}(k \log^2 n)$	$\mathcal{O}(1)$